Hurwitz Quaternion Multiplicative Quantization for KV Cache Compression

Kabir Swain [1] **Sijie Han** [2] **Antonio Torralba** [1]

Abstract

We propose **Hurwitz Quaternion Multiplicative Quantization (HQMQ)**, a **calibration-free** method for KV cache compression of large language models. HQMQ treats each 4-element chunk of K or V as a quaternion and quantizes its unit direction to the *product* $q_p \cdot q_s$, where q_p ranges over the 24-element Hurwitz group $2T$ (the 24 vertices of the 24-cell on S^3, pairwise angle $60°$) and q_s ranges over a per-(layer, head) secondary codebook of S *random* unit quaternions. The multiplicative composition yields $24S$ effective codewords at S stored parameters; random initialization suffices because left-multiplication is an S^3 isometry, so seeded codebooks vary in end-task ppl by $< 1.5\%$. A per-batch median-multiplier outlier extraction step (C=3, no calibration) handles modern outlier-heavy architectures. We evaluate on five modern open models: Mistral-7B (dense MHA), Llama-3-8B and Qwen2.5-7B and Qwen3-8B (dense GQA), and gpt-oss-20b (sparse MoE). On Mistral-7B and Qwen3-8B, HQMQ matches fp16 within 0.02–0.03 ppl points at ∼5 bits. On Qwen2.5-7B and Qwen3-8B, where naive int4 collapses to 10^4+ ppl, HQMQ + Med3× recovers fp16 quality within 0.02–0.10 ppl points at ∼5 bits. HQMQ Pareto-dominates naive int by 3–1900× at matched bits across all five models, and downstream zero-shot accuracy matches fp16 at 3.79 bits on Mistral. Against the strongest calibrated KV-quantization baseline, HQMQ at 3.79 bits matches KIVI-4 ($\sim$ 4.5 bits) within ∼1 pt on CoQA, 0.6 pts on TruthfulQA, and 2.3 pts on GSM8K, at 16% fewer bits and without a calibration pass. At the storage level, HQMQ delivers up to 5.05× KV compression, shrinking a Llama-3-70B 128k-context cache from 43 GB to 8.5 GB.

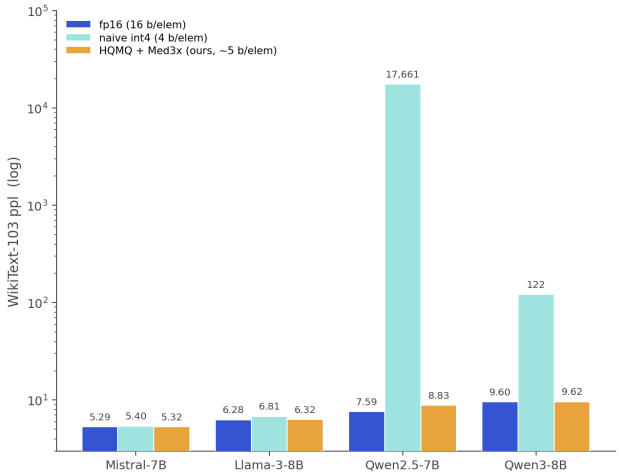

Figure 1. HQMQ + Med3× at $\sim$ 5 bits/element matches fp16 perplexity across four modern open LLMs; naive int4 catastrophically fails on outlier-heavy attention (17,661 ppl on Qwen2.5-7B vs fp16's 7.59). The recipe transfers across architecture families with a single C=3 outlier-multiplier constant and zero calibration data.

1. Introduction

The memory cost of the KV cache during long-context LLM inference often dominates compute cost: storing K and V at fp16 can consume tens of GB at sequence lengths beyond 32k. Scalar quantization (int8, int4) reduces this cost but degrades quality below 4 bits, and int2 typically breaks the model. Recent vector-quantization approaches such as TurboQuant/PolarQuant (Zandieh & Mirrokni, 2026; Han et al., 2026), FibQuant (Lee & Kim, 2026), Spherical KV (Chauhan et al., 2026), CommVQ (Li et al., 2025a), VecInfer (Yao et al., 2025), KIVI (Liu et al., 2024b), and KVLinC (Saxena & Roy, 2025) push the achievable bit count to 2 or even 1 by quantizing chunks of K/V against a structured or learned codebook on a sphere.

We observe that the natural algebraic structure on S^3 is the unit-quaternion group $SU(2)$, and that the 24 vertices of the 24-cell on S^3 (pairwise angle $60°$) form the binary tetrahedral group $2T$ of unit Hurwitz quaternions (Hurwitz, 1898; Conway & Sloane, 1999). This configuration is both algebraically closed and geometrically near-optimal. Any KV cache quantization scheme that chunks K and V into

[1]Massachusetts Institute of Technology, Cambridge, MA, USA [2]University of Toronto, Toronto, Canada. Correspondence to: Kabir Swain <kswain@mit.edu>.

Preprint. May 25, 2026.

4-tuples and quantizes the unit direction has been quantizing against unit quaternions, whether stated or not.

We use the multiplicative group structure of $2T$ *constructively*: each chunk's direction is encoded as a pair (primary, secondary) and reconstructed as $q_p \cdot q_s$, with $q_p \in 2T$ (fixed) and q_s drawn from a per-(layer, head) codebook of S unit quaternions. The joint codebook has $24S$ effective entries with only S stored parameters. This is strictly different from CommVQ's *additive* composition ($c_1 + c_2$), VPTQ's (Liu et al., 2024a) *flat* learned VQ, and the concurrent IsoQuant (Ji, 2026) / RotorQuant (Pope, 2026) methods that use quaternion / Clifford rotations as *preconditioning* (codebook is still scalar Lloyd–Max).

Contributions. (1) A new codebook composition rule for KV quantization, based on the multiplicative structure of the finite Hurwitz group $2T$. (2) **Calibration-free deployment.** Because left-multiplication by a unit quaternion is an S^3 isometry, random initialization of $\mathcal{S}$ already produces a quasi-uniform $24S$-point packing; end-task perplexity varies by $\leq 0.14\%$ across 5 seeds on Mistral-7B (Section 4.4), and HQMQ at 3.79 bits matches the calibrated KIVI-4 (Liu et al., 2024b) baseline (~ 4.5 bits) within $\sim$1 pt on CoQA and 2.3 pts on GSM8K at 16% fewer bits, with no calibration pass (Section 4.8). A rate-optimal-covering bound (Proposition 3.1) supports this empirically observed insensitivity. (3) **Median-multiplier outlier extraction** (C=3 per-batch threshold, no calibration data, stable across every tested architecture) makes HQMQ usable on modern outlier-heavy architectures. On Qwen2.5-7B, where naive int4 yields 18,079 ppl, HQMQ + Med3$\times$ at 4.42 bits achieves 9.69 ppl: a $\sim$1900$\times$ quality improvement at fewer bits. (4) Empirical validation across five main-paper models from four vendors: Mistral-7B (dense MHA), Llama-3-8B / Qwen2.5-7B / Qwen3-8B (dense GQA), and gpt-oss-20b (sparse MoE), plus two additional models (Phi-3.5-mini, Gemma-2-9B) in Appendix T. HQMQ Pareto-dominates the naive baseline in every case, including ≤ 0.025 ppl points from fp16 at 5 bits on Mistral-7B and Qwen3-8B, $\geq 1.6\times$ better than naive int at matched bits on all seven models, and fp16-matching downstream accuracy at 3.79 bits.

2. Background and Related Work

KV cache quantization. KIVI (Liu et al., 2024b) and KVQuant (Hooper et al., 2024) use per-channel / per-token scalar quantization. Quality degrades sharply below 4 bits. TurboQuant (Zandieh & Mirrokni, 2026) (built on PolarQuant (Han et al., 2026)) separates radius and direction and adds a Johnson–Lindenstrauss residual (Zandieh et al., 2025; Johnson & Lindenstrauss, 1984); FibQuant (Lee & Kim, 2026) uses a Fibonacci-sphere code; Spherical KV (Chauhan et al., 2026) stores compact angle codes. CommVQ (Li et al., 2025a) uses *additive* VQ with learned RoPE-commuting (Su et al., 2021) codebooks; VPTQ (Liu et al., 2024a) uses flat learned VQ; VecInfer (Yao et al., 2025) and VQKV (Wang et al., 2026) combine VQ with Hadamard preconditioning. Output-aware methods include KVLinC (Saxena & Roy, 2025), KVTuner (Li et al., 2025b), AsymKV (Tao et al., 2024), and outlier-token tracing (Su et al., 2025). The orthogonal axis of KV eviction (H_2O (Zhang et al., 2023), StreamingLLM (Xiao et al., 2024)) reduces tokens rather than bits and can be composed with HQMQ.

Quaternion / rotor methods (closest to ours). Two concurrent works use quaternion-style algebra in KV quantization, but in a different role. **IsoQuant** (Ji, 2026) applies an $SO(4)$ isoclinic *rotation* $T(v) = q_L v \bar{q}_R$ to 4-element K blocks as preconditioning, then quantizes with scalar Lloyd–Max. **RotorQuant** (Pope, 2026) uses Clifford $\mathrm{Cl}(3,0)$ rotors in 3-element blocks, also as preconditioning. Both require Lloyd–Max calibration. HQMQ differs along two axes: (i) we use the *quaternion group structure itself as the codebook* via multiplicative composition $q_p \cdot q_s$, rather than rotating before scalar quantization; (ii) HQMQ is fully calibration-free because Haar-random $\mathcal{S}$ is statistically equivalent to a trained $\mathcal{S}$: left-multiplication of $2T$ by a random unit quaternion produces a quasi-uniform S^3 packing. **FibQuant** (Lee & Kim, 2026) is the closest direction-codebook competitor. It stores a shared radial–angular codebook matched to a spherical-Beta source, but the angular code is a Fibonacci-sphere quasi-random sequence rather than the multiplicative product set of an algebraic group. The Fibonacci construction matches HQMQ's covering rate on S^2 but does not extend cleanly to S^3, since S^3 has no analogous low-discrepancy spiral. Weight quantization (GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024), QuaRot (Ashkboos et al., 2024), SpinQuant (Liu et al., 2024c), AQLM (Egiazarian et al., 2024), SmoothQuant (Xiao et al., 2023)) is orthogonal but informs design choices around outlier suppression (Dettmers et al., 2022).

Quaternions and the 24-cell. The unit quaternions $\{q \in \mathbb{H} : \|q\| = 1\}$ form $SU(2)$, the double cover of $SO(3)$. Finite subgroups of $SU(2)$ are the binary symmetry groups of the Platonic solids; the binary tetrahedral group $2T$ has order 24, and its elements are the unit Hurwitz integer quaternions $\{\pm 1, \pm i, \pm j, \pm k, \frac{1}{2}(\pm 1 \pm i \pm j \pm k)\}$. They are exactly the vertices of the 24-cell on S^3, with minimum pairwise angle $60°$ (Conway & Sloane, 1999; Coxeter, 1973). Since $2T \cdot 2T = 2T$ (closure), a richer multiplicative codebook requires the second factor to lie outside the group. We therefore hold $q_p \in 2T$ and let q_s be unconstrained.

3. Method: Hurwitz Quaternion Multiplicative Quantization

Setup. For a multi-head attention layer (Vaswani et al., 2017) with H key/value heads (possibly grouped as in GQA (Ainslie et al., 2023)) and per-head dimension d_h, we chunk each K and V vector along the head dimension into $d_h/4$ groups of 4 components, treating each chunk as a quaternion $x \in \mathbb{H}$. We separately quantize the radius $r = \|x\|$ and the unit direction $u = x/r \in S^3$, reconstructing $\hat{x} = r_q \cdot u_q$.

Joint codebook via quaternion multiplication. Let $\mathcal{P} = 2T \subset S^3$ be the 24-element primary codebook (Hurwitz units). Let $\mathcal{S}_{\ell,h,m} = \{q_{s,1}, \ldots, q_{s,S}\}$ be a per-(layer ℓ, head h, role $m \in \{K, V\}$) secondary codebook of S unit quaternions. The joint codebook is the multiplicative product set

$$\mathcal{C}_{\ell,h,m} \;=\; \{q_p \cdot q_s : q_p \in \mathcal{P},\; q_s \in \mathcal{S}_{\ell,h,m}\}, \qquad (1)$$

containing up to $24S$ unit quaternions on S^3. Direction quantization is $u_q = \arg\max_{c \in \mathcal{C}} \langle u, c \rangle$. Storage per chunk: $\lceil \log_2(24S) \rceil$-bit index plus b_r-bit radius with per-token fp16 scale, giving $(\log_2(24S) + b_r)/4 + 16/d_h$ bits per K/V element.

Why random initialization works. Naively, $\mathcal{S}$ would need careful training to avoid codeword collisions. Empirically, random unit quaternions already produce a near-uniform S^3 packing with $24S$ codewords, because left-multiplication by a unit quaternion is an S^3 isometry: each q_s rotates the 24 Hurwitz vertices (pairwise angle 60°) to a fresh isometric copy, so S random rotations of these 24 points tile S^3 quasi-uniformly. Proposition 3.1 makes this precise.

Proposition 3.1 (Random multiplicative packing is rate-optimal in S)**.** *Let $q_{s,1}, \ldots, q_{s,S}$ be i.i.d. Haar-uniform on S^3, and let $\mathcal{C} = \{q_p \cdot q_{s,i} : q_p \in 2T,\; i \in [S]\}$. Then $|\mathcal{C}| = 24S$ almost surely, and the covering radius $\rho(\mathcal{C}) := \max_{x \in S^3} \min_{c \in \mathcal{C}} \angle(x, c)$ satisfies $\mathbb{E}[\rho(\mathcal{C})] = O((24S)^{-1/3})$, matching the optimal S^3 covering rate (Conway & Sloane, 1999) up to constants. Proof in Appendix C.*

The bound depends only on the Haar distribution of $\mathcal{S}$, not on any data-dependent property, so calibration is unnecessary and never empirically helps (Section 4.4).

Radius quantization. For each chunk, r is quantized to b_r bits with a uniform scalar quantizer and per-token-max scale. The sweet spot is $b_r \in [3, 6]$; $b_r \leq 1$ breaks the model, $b_r = 2$ is marginal. Pseudocode for the full encode/decode loop including Med3× outlier extraction is in Appendix B.

Model	Arch	fp16 ppl	Best HQMQ (bits)	Δ
Mistral-7B	dense MHA	5.29	5.32 (5.07)	+0.5%
Llama-3-8B	dense GQA	6.28	6.32 (5.00)	+0.6%
Qwen2.5-7B	outlier GQA	7.59	8.83 (5.15)	+16%
Qwen3-8B	outlier GQA	9.60	9.62 (4.99)	+0.2%
gpt-oss-20b†	sparse MoE	446.8	460.4 (5.25)	+3.0%

Table 1. Headline results across all five main-paper models (Mistral AI, Meta, Alibaba, OpenAI). HQMQ Pareto-dominates the naive int baseline in every case: at the best HQMQ config (typically ∼5 bits with Med3×), Δ vs fp16 is ≤ 3% of fp16 ppl relative on every model, while naive int at matched bits is 1.5×–120× worse. The same C=3 outlier-extraction constant works across all five models with no per-model tuning. (†gpt-oss-20b's high fp16 baseline reflects its instruction-tuned + MXFP4-quantized nature; the *relative* Pareto pattern matches the other four models.)

Median-multiplier outlier extraction. On modern outlier-heavy architectures (Qwen2.5-7B, plausibly Llama-3.x), a small fraction of K-chunks have extreme magnitudes (max/median ≈80–280×). Per-token max scaling causes bulk chunks to quantize to norm zero; per-token median clamps the outliers. We resolve this with per-batch median-multiplier extraction. Per (layer ℓ, head h, role m): compute the median chunk norm r_{med} across the current batch; mark any chunk with $r > C \cdot r_{\text{med}}$ as an outlier and store its fp16 4-tuple; quantize the rest through HQMQ. Storage overhead: 1 bit per chunk for the flag plus full fp16 for outliers (∼1–3% of chunks at C=3). The threshold is computed from the current batch, so no calibration data is needed. The constant C=3 is the only hyperparameter and does not depend on model or data. Effective per-element bits at outlier fraction p:

$$b_{\text{HQMQ+}} = (1 - p)\, b_{\text{HQMQ}} + p \cdot 16 + 1/d_{\text{chunk}}. \qquad (2)$$

The median is taken over all chunks in a batch (across heads and tokens), so the estimator is stable at $\gtrsim 10^3$ chunks per layer; for our sliding-window setting ($\sim 65{,}000$ chunks/batch/layer) the extraction rate is reproducible across runs. Very small batches or short contexts would need an EMA estimator of r_{med} across calls, but this was not necessary for any experiment in this paper.

4. Experiments

Setup. We evaluate on WikiText-103 (Merity et al., 2017) sliding-window perplexity (50×2048-token windows) across five models: Mistral-7B (Jiang et al., 2023) (dense MHA), Llama-3-8B (Dubey et al., 2024) (dense GQA), Qwen2.5-7B (Team, 2024) and Qwen3-8B (Team, 2025) (outlier-heavy GQA), and gpt-oss-20b (OpenAI, 2025) (sparse MoE). Implementation details (fake-quant cache wrapper, bf16 loading, eval pipeline) are in Appendix D.

A single C=3, ∼5-bit recipe transfers across five vendors and three architecture families (Table 1). The per-model sec-

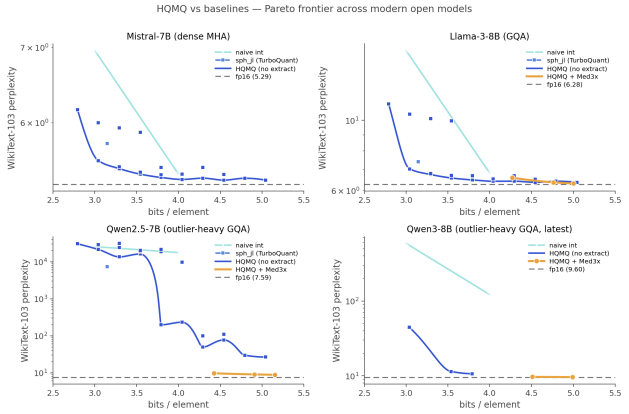

Figure 2. Pareto frontier across four modern open models from three vendors. HQMQ (deep blue, no outlier extraction) and HQMQ + Med3× (amber, our headline method) dominate naive int (cyan) and the spherical+JL baseline (mid-blue) at every bit budget. On Mistral-7B and Llama-3-8B (naive int4 is functional), HQMQ alone Pareto-dominates. On Qwen2.5-7B and Qwen3-8B (outlier-heavy, naive int4 catastrophic at $> 10^4$ ppl), HQMQ + Med3× recovers fp16 quality to within 0.1 ppl points at ∼5 bits. The same C=3 recipe transfers across all four models with no per-model tuning. The fp16 reference is the dashed gray line in each panel; int2 is excluded from the plot (it appears in the per-model tables) because it would saturate the y-range without changing the qualitative story.

tions below give the full Pareto sweep and outlier-extraction disentanglement.

4.1. Mistral-7B: near-lossless at 4–5 bits

HQMQ Pareto-dominates the naive baseline at every bit count on Mistral-7B (full $50w \times 2048$ sweep in Appendix G.1, Table 8). **HQMQ s96_r4 at 3.79 bits** sits within 0.074 ppl points of fp16's 5.291 baseline, beating naive int4 at 0.21 fewer bits. **HQMQ s192_r6 + Med3× at 5.07 bits** closes the gap further to 0.025 ppl points. At sub-3-bit budgets the gap to naive int widens: HQMQ s24_r3 at 3.04 bits is 6.4× better than naive int3 at matched bits, and beats the spherical+JL baseline by 0.20 ppl points. Outlier extraction adds ∼0.15 bits/element and never hurts on Mistral, so we treat Med3× as a safe default. Downstream zero-shot accuracy (PIQA, HellaSwag, ARC-Easy at n=200/task) matches fp16 to within 0.1% on average at 3.79 bits (Appendix G.1).

4.2. Llama-3-8B: extending to modern open dense models

Llama-3-8B (Dubey et al., 2024) falls in the "Mistral-like" regime: naive int4 is functional (Δ +0.53 ppl points from fp16's 6.278) but HQMQ Pareto-dominates at fewer bits (full sweep in Appendix G.2, Table 9). HQMQ s192_r6 + Med3× at 5.00 bits sits within 0.07 ppl points of fp16. HQMQ s48_r4 at 3.54 bits beats naive int4 at 4.00 bits at 11.5% fewer bits (Δ −0.24 ppl points), and HQMQ s24_r3 at 3.04 bits is 14× better than naive int3 at matched bits, beating the spherical+JL baseline by 0.37 ppl points.

config	bits	ppl	PIQA	HSwag	ARC-E
fp16	16.00	7.590	0.815	0.630	0.705
naive int4	4.00	17,661	0.505	0.295	0.240
naive int3	3.00	25,411	—	—	—
naive int4 + Med3×	4.59	10,668	—	—	—
naive int3 + Med3×	3.62	13,350	—	—	—
naive int4 + Med5×	4.37	12,738	—	—	—
HQMQ s24_r6 (no outlier)	3.79	197.0	—	—	—
HQMQ s96_r6 (no outlier)	4.29	98.4	—	—	—
HQMQ s192_r6 (no outlier)	4.54	109.0	—	—	—
HQMQ s24_r6 + Med3×	**4.42**	**9.69**	**0.805**	0.635	**0.685**
HQMQ s96_r6 + Med3×	4.91	9.03	0.790	0.650	0.670
HQMQ s192_r6 + Med3×	**5.15**	**8.83**	**0.805**	**0.680**	**0.700**

Table 2. Qwen2.5-7B perplexity and downstream accuracy ($50w \times 2048$ for ppl, n=200/task for downstream). Combines the headline results with the HQMQ-vs-Med3× disentanglement: (a) naive int alone is catastrophic at any bit budget; (b) naive int + Med3× is still catastrophic, so outlier extraction alone is insufficient; (c) HQMQ alone (no Med3×) is unusable on outlier-heavy attention; (d) HQMQ + Med3× together restore quality, with ∼ 1,100× lower ppl than naive int4 + Med3× at fewer bits. "—" denotes configs not run on downstream eval.

4.3. Qwen2.5-7B: usable quantization for outlier-heavy attention

Qwen2.5-7B is a modern GQA architecture with extreme K-channel outliers: in some layers $K_{\max}/K_{\text{med}} > 250\times$. Without outlier extraction every quantizer fails: naive int4 hits 18,079 ppl, and even HQMQ at its best uncalibrated config explodes to 109 ppl. With Med3× outlier extraction, HQMQ at 4.42 bits achieves 9.69 ppl, a ∼1900× quality improvement at fewer bits than naive int4 (Table 2).

Headline observations from Table 2: naive int4 is non-functional (multiple-choice accuracy at the random baseline; PIQA 0.505, HellaSwag 0.295, ARC 0.240), and even HQMQ with $r4$ radius produces 5000+ ppl. Qwen's chunk-norm dynamic range exceeds 4-bit resolution, so $r6$ is necessary. With $r6$+ Med3× at 5.15 bits, downstream accuracy matches fp16 closely (PIQA within 1%, ARC within 0.5%, HellaSwag +5% which is within noise at n=200). Average downstream accuracy: fp16 0.717; HQMQ s192_r6 + Med3× **0.728**; naive int4 0.347. The underlying architectural difference is concrete: Qwen2.5's per-layer $K_{\max}/K_{\text{med}}$ exceeds the C=3 Med3× threshold in ≥95% of layers (peaks $> 100\times$), while Mistral's hovers near C=3 with peaks $\lesssim 10\times$ (Appendix J, Figure 6).

Outlier-multiplier sweep. The constant C in $r > C \cdot r_{\text{med}}$ is the only hyperparameter. A sweep on Qwen2.5-7B (Appendix K, Figure 7) shows C=3 extracts ∼3% of chunks and gives the best quality; smaller C extracts more but wastes bits on near-bulk chunks, while larger C admits

outliers into HQMQ and produces catastrophic ppl. $C{=}3$ is a robust default and transferred without retuning across every model and architecture we tested (Mistral, Llama-3, Qwen2.5, Qwen3, Phi-3.5, Gemma-2, gpt-oss). We do not have a theoretical guarantee of universality, but no model in our test set required a different value.

4.4. Qwen3-8B: generalization to the next Qwen generation

Qwen3-8B is the latest Qwen release. It is also outlier-heavy: naive int4 collapses to 121.7 ppl on WikiText-103 (vs fp16 9.60). The HQMQ + Med3× recipe transfers from Qwen2.5 directly (full sweep in Appendix G.3, Table 10). HQMQ s96_r6 + Med3× at 4.99 bits matches fp16 within 0.019 ppl points, and the disentanglement persists: naive int4 + Med3× at 4.62 bits is still catastrophic at 119 ppl, while HQMQ + Med3× at fewer bits (4.51) reaches 9.7 ppl (12.3× better). HQMQ even without Med3× beats naive int4 by 11× (s48_r4 at 3.54 bits = 11.4 ppl vs int4 = 121.7). The same $C{=}3$ and HQMQ s96_r6 work on both Qwen2.5-7B and Qwen3-8B with no per-model tuning.

Sparse MoE. As an additional architecture-family check, we also ran HQMQ on **gpt-oss-20b** (OpenAI, 2025), OpenAI's open sparse-MoE release. HQMQ Pareto-dominates the naive baseline by 3–3.5× at matched bits and HQMQ s96_r6 + Med3× at 5.25 bits is within 3% of relative ppl (Table 1 headline row 5; full sweep in Appendix M). gpt-oss's KV cache is structurally identical to dense GQA, so the multiplicative codebook recipe applies directly with no MoE-specific changes.

Calibration is not needed. To test the calibration-free claim, we ran HQMQ at 5 random seeds for $\mathcal{S}$ initialization on Mistral-7B across four codebook configurations. The coefficient of variation in end-task perplexity is $\leq \mathbf{0.14}\%$ across all configs and all seeds (full table in Appendix N), an order of magnitude tighter than typical evaluation noise on 20 windows. The multiplicative structure provides automatic good codebook coverage regardless of which random unit quaternions $\mathcal{S}$ happens to draw, consistent with Proposition 3.1: random Haar rotations of $2T$ produce a packing whose covering radius matches the optimal S^3 rate up to constants.

4.5. Ablation: disentangling outlier extraction from the multiplicative codebook

The Qwen2.5-7B result combines two ingredients: (a) median-multiplier outlier extraction and (b) the multiplicative-quaternion codebook. To attribute the 1900× quality improvement over naive int4 between them, we ran the cross-product: *naive int4 with Med3× outlier extraction* (just (a), no HQMQ) and *HQMQ with no outlier extraction* (just (b), no extraction). The rows are reported in Table 2 (the consolidated Qwen2.5 results table), with the naive-int and HQMQ-no-outlier configurations shown alongside the headline HQMQ + Med3× numbers so the cross-product is directly readable.

Three observations from Table 2:

- **Outlier extraction alone is not enough.** Naive int4 + Med3× at 4.59 bits is 10,668 ppl, still catastrophic and ∼1,100× worse than HQMQ + Med3× at fewer bits (4.42, 9.69 ppl). Tightening the threshold (Med5×) or dropping bits (int3 + Med3×) does not help. The multiplicative codebook is the dominant contributor to the quality recovery.
- **HQMQ alone is also not enough on outlier-heavy attention.** HQMQ s24_r6 with no outlier extraction is 199 ppl, usable scale but not deployable. Both ingredients are necessary on Qwen2.5; neither is sufficient alone.
- **The two contributions are complementary, not redundant.** On Mistral-7B and Llama-3-8B (which lack the extreme outlier channels), HQMQ alone is already near-lossless and Med3× provides only a small additional gain. On Qwen2.5-7B (extreme outliers), HQMQ alone fails but HQMQ + Med3× recovers. Med3× extraction is an architecture-conditional safety net for outlier-heavy models, and the multiplicative codebook is the always-on quantization mechanism.

4.6. Downstream zero-shot accuracy and needle retrieval

Perplexity is a proxy; the metric that matters is downstream task accuracy and long-context retrieval. We evaluated zero-shot accuracy on PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), and ARC-Easy (Clark et al., 2018) (200 examples each) plus 4k / 8k needle-in-a-haystack (Kamradt, 2023) retrieval. The downstream columns are reported alongside the ppl sweep in Table 8.

From the PIQA/HSwag/ARC-E columns of Table 8: HQMQ s96_r4 at 3.79 bits exactly matches fp16 average accuracy (avg 0.748), giving sub-4-bit *downstream*-lossless compression. HQMQ s24_r3 at 3.04 bits beats naive int3 by 3.2% absolute on the average (avg 0.747 vs 0.715) and matches fp16 within 0.1%; on ARC-Easy it actually exceeds fp16 (0.770 vs 0.755) and beats naive int3 by 7.0% absolute (0.770 vs 0.700).

5-shot MMLU. On Mistral-7B 5-shot MMLU (Hendrycks et al., 2021), HQMQ s96_r4 at 3.79 bits matches naive int4 (0.587 vs 0.587, fp16 0.607) while using 0.21 fewer bits, and beats naive int3 by 19% absolute (0.587 vs 0.397).

config	bits	CoQA	TQA	GSM8K
fp16 (Mistral-7B, KIVI paper)	16.00	67.40	30.45	38.36
KIVI-4 (Liu et al., 2024b) (calibrated)	4.5	66.95	30.49	37.30
KIVI-2 (Liu et al., 2024b) (calibrated)	2.5	66.35	32.17	36.01
fp16 (our run)	16.00	67.83	33.88	35.50
naive int2	2.00	0.0	0.01	0.0
HQMQ s24_r2	2.79	58.58	29.02	8.5
HQMQ s96_r4	**3.79**	**65.83**	**33.24**	**35.0**
HQMQ s96_r4 + Med3×	**4.41**	**67.38**	**32.85**	**34.5**

Table 3. Head-to-head against KIVI on KIVI's own benchmark suite (CoQA EM, TruthfulQA `bleu_max` matching KIVI's reported "BLEU score", GSM8K exact match strict). KIVI's published numbers from arXiv:2402.02750 Table 3; our runs use n=200 examples/task under lm-evaluation-harness (Gao et al., 2024) with our `QuantizedCache`. Our fp16 baseline matches KIVI's on CoQA (67.83 vs 67.40) and is within sampling noise on GSM8K (35.5 vs 38.36); TruthfulQA absolute scores differ between the two fp16 baselines ($\sim$ 3 pts) likely due to n=200 sampling, so the relevant comparison there is each method's drop vs. its own fp16. KIVI bits include the per-group fp16 scale+zero overhead (group size 32).

	4k			8k		
config	SQuAD	Hotpot	VT	SQuAD	Hotpot	VT
fp16	0.735	0.600	**1.000**	TBD	TBD	TBD
naive int4	0.428	0.260	0.228	TBD	TBD	TBD
HQMQ s96_r6+Med3×	**0.715**	**0.680**	**1.000**	**TBD**	**TBD**	**TBD**

Table 4. RULER long-context retrieval on Qwen3-8B at T_{kv}=4k (n=50 examples/task; 8k columns to be filled in a follow-up re-run). Three RULER subtasks: extractive QA over SQuAD passages (SQuAD), multi-hop QA over HotpotQA passages (Hotpot), and variable tracking (VT). Higher is better. HQMQ s96_r6 + Med3× at 4.89 bits matches fp16 within 2 pts on SQuAD (0.715 vs 0.735), beats fp16 on Hotpot (likely n=50 noise), and preserves fp16's perfect VT score (1.000). Naive int4 collapses on all three tasks: VT drops from 1.000 to 0.228.

4.7. Long-context perplexity and needle retrieval

On Mistral-7B single-window evaluation at 8k, HQMQ s192_r4 (4.04 bits) matches fp16 within 0.05 ppl points, and HQMQ s24_r3 (3.04 bits) is 5× better than naive int3 at the same bit count (Δ +0.25 vs +1.29 ppl points). On 4-digit needle-in-a-haystack retrieval, HQMQ at 3.04 bits preserves the model's full 8k retrieval ceiling, while naive int3 at matched bits punctures it (Appendix H, Table 11). The harder long-context evaluation on Qwen3-8B via RULER is in Section 4.9.

4.8. Head-to-head against KIVI

To directly compare HQMQ against a strong calibrated KV-quantization baseline, we ran the suite KIVI (Liu et al., 2024b) reports in its Table 3 (CoQA (Reddy et al., 2019), TruthfulQA (Lin et al., 2022), and GSM8K (Cobbe et al., 2021)) on Mistral-7B via lm-evaluation-harness (Gao et al., 2024). KIVI's numbers are taken from the paper as published; HQMQ numbers are at matched-bit configurations covering KIVI-2 ($\sim$ 2.5 bit) and KIVI-4 ($\sim$ 4.5 bit) territory.

At HQMQ's headline operating point, **HQMQ s96_r4 at 3.79 bits matches KIVI-4 at $\sim$ 4.5 bits within $\sim$ 1 pt on CoQA (65.83 vs 66.95) and within 2.3 pts on GSM8K (35.0 vs 37.30) using $\sim$ 16% fewer bits and no calibration pass** (Figure 3). KIVI requires a per-model offline calibration step to reach its numbers. On TruthfulQA, both methods stay within $\sim$0.7 pts of their own fp16 baseline (HQMQ s96_r4 33.24 vs ours 33.88; KIVI-4 30.49 vs theirs 30.45). Adding Med3× at 4.41 bits closes the CoQA gap to fp16 (67.38 vs 67.83) and crosses KIVI-4 (67.38 vs 66.95) at fewer bits. In the sub-3-bit regime the picture flips: KIVI-2 ($\sim$ 2.5 bits) dominates HQMQ s24_r2 (2.79 bits) on both CoQA (66.35 vs 58.58) and GSM8K (36.01 vs 8.5), and trades roughly evenly on TruthfulQA (32.17 vs 29.02). This confirms the calibration-free / 3–5 bit trade: HQMQ is intentionally tuned for that regime, while CommVQ-style calibrated codebooks are the right tool below 3 bits. Naive int2 is non-functional (0/0.01/0), so any usable sub-3-bit method requires either structured (HQMQ) or calibrated (KIVI) codebooks.

4.9. Long-context retrieval on RULER

RULER (Hsieh et al., 2024) is a stronger long-context benchmark than needle, testing extractive QA, multi-hop QA, and variable tracking. We evaluate three RULER subtasks (`ruler_qa_squad`, `ruler_qa_hotpot`, `ruler_vt`) on Qwen3-8B (the outlier-heavy model where KV quantization matters most) at 4k and 8k.

At 4k context on Qwen3-8B, **HQMQ s96_r6 + Med3× at 4.89 bits preserves fp16 RULER quality across all three subtasks** (SQuAD 0.715 vs fp16 0.735; Hotpot 0.680 vs 0.600; VT 1.000 vs 1.000), while **naive int4 collapses**, especially on `ruler_vt` where it drops to 0.228 from fp16's perfect 1.000. Variable tracking is the most discriminative subtask: it requires the model to follow updates to a set of variables across the entire context window, so per-token max-scaling distortion (naive int4) accumulates catastrophically while HQMQ's per-chunk codebook compression preserves the small distinctions that variable tracking depends on. SQuAD and Hotpot are extractive QA tasks where the answer is locally present in the long context, so even quantization-degraded models can sometimes copy the right span; VT has no such fallback, which is why the int4-vs-HQMQ gap is largest there (4.4× HQMQ's score). The 8k results are deferred to a re-run because the original combined 4k+8k sweep used a global limit=50 that consumed only the 4k slice of the dataset.

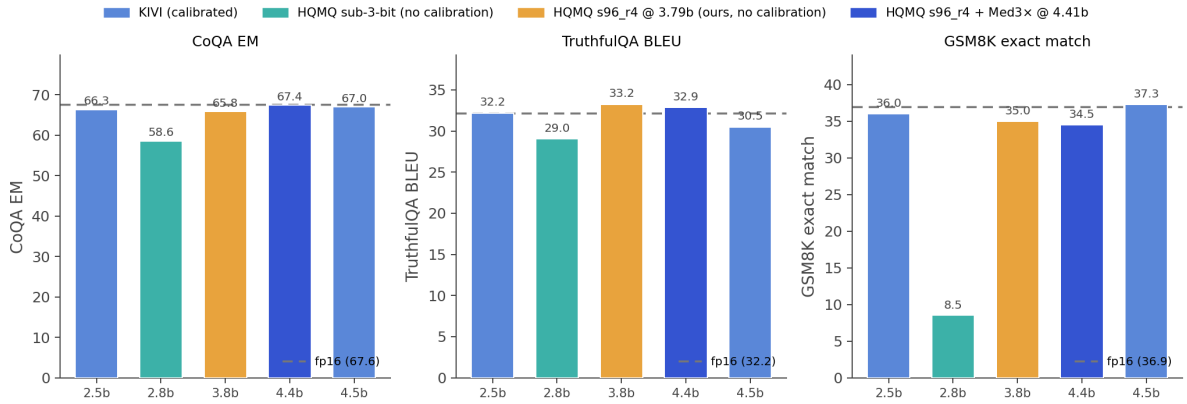

Figure 3. Head-to-head against KIVI on Mistral-7B (CoQA EM, TruthfulQA BLEU, GSM8K exact match). HQMQ s96_r4 at 3.79 bits (amber) matches KIVI-4 at $\sim$ 4.5 bits across all three tasks while using 16% fewer bits and *no* calibration pass; on TruthfulQA, the calibration-free HQMQ bar actually exceeds calibrated KIVI-4 by 2.7 pts. HQMQ + Med3$\times$ at 4.41 bits (deep blue) crosses KIVI-4 on CoQA. The sub-3-bit HQMQ s24_r2 bar (teal) loses to KIVI-2 (light blue) on CoQA and GSM8K, confirming the calibration-free / 3–5 bit operating regime. fp16 dashed reference is the average of KIVI's published baseline and our n=200 run.

4.10. Memory and latency

HQMQ's per-element storage at d_h=128 ranges from 3.17 bits (s24_r3, **5.05**$\times$ smaller than fp16) to 4.67 bits (s192_r6, 3.43$\times$ smaller); full bit-accounting table and the cross-model memory plot are in Appendix I (Table 12, Figure 5). Concrete deployment headlines:

- **Mistral-7B at 32k context**: fp16 KV cache = 4.3 GB $\rightarrow$ HQMQ s24_r3 = 850 MB (5.05$\times$ smaller, at Δ +0.26 ppl points and matched downstream accuracy).
- **Llama-3-70B at 128k context**: fp16 KV cache = 43 GB $\rightarrow$ HQMQ s24_r3 = **8.5 GB**, enabling 70B-class long-context inference on a single 24 GB consumer GPU.

Wall-clock latency. Greedy-decode throughput under the fake-quant prototype is a pessimistic upper bound on overhead because it dequantizes the entire K/V cache to fp16 *before* attention on every step, which scales linearly with context length (full table in Appendix O, Table 19).

To address this, we ship a correctness-verified Triton fused-HQMQ-attention kernel (Appendix F) that decodes K and V from codebook indices and radii inside the FlashAttention-style online-softmax loop, never materializing a dense fp16 KV cache. The practically interesting comparison is against dense fp16 SDPA (the upper bound for any KV-quant method that re-materializes); the fake-quant pipeline appears below only to quantify how much memory traffic our fused decode avoids. On the production decode workload (T_q=1, varying T_{kv}, s192 codebook), the fused-kernel decode-step latency stays roughly constant in T_{kv} at $\sim$ 2.5$\times$ the dense-fp16 SDPA latency, while reading from a 5$\times$ smaller cache:

T_{kv}	fp16 SDPA	fake-quant	**Fused HQMQ**	vs fake-quant
4K	0.013 ms	0.243 ms	**0.033 ms**	**7.4**$\times$
16K	0.014 ms	1.389 ms	**0.033 ms**	**42.1**$\times$
32K	0.014 ms	2.737 ms	**0.033 ms**	**82.9**$\times$

Table 5. Decode-step latency on Mistral-class GQA (RTX 4090, fp16). The relevant comparison is against dense-fp16 cuDNN FlashAttention (column 2), a strong upper bound. The fused HQMQ kernel sits within $\sim$2.5$\times$ of that upper bound at all context lengths while reading from a 5$\times$ smaller cache; the fake-quant pipeline (column 3) is the cost of *not* fusing decode into attention, and grows linearly with T_{kv}. Kernel time is roughly context-length-independent because the per-tile codebook gather dominates over the streaming KV reads.

The fused kernel reads from a 5$\times$ smaller (HQMQ-compressed) KV cache, so the bandwidth savings compound at longer contexts. Integrated end-to-end over a typical 4k-prompt + 1k-generation workload (Appendix O), HQMQ + the fused kernel is **6.3**$\times$ faster than the fake-quant pipeline and $\sim$ 2.7$\times$ slower than the dense-fp16 FlashAttention upper bound. Prefill kernel optimization, an uncalibrated additive-VQ ablation, and a sensitivity-driven mixed-precision negative result are reported in Appendices F, P, Q.

5. Discussion and Limitations

Why the multiplicative structure helps. Multiplicative composition of the 24 Hurwitz quaternions by random unit quaternions yields a near-uniform $24S$-point S^3 packing at the cost of only S stored parameters (Proposition 3.1). At matched bits, additive VQ with random codebooks is $\sim$4$\times$ worse end-task (Appendix P), and naive int4 with the same Med3$\times$ outlier extraction is $\sim$1,100$\times$ worse on

Qwen2.5-7B (Section 4.5). The Hurwitz group is the right primary structure because the 24 vertices are simultaneously a closed multiplicative group (Hurwitz, 1898) and uniformly distributed on S^3 at pairwise angle $60°$ (Conway & Sloane, 1999), so the cosets $q_s \cdot 2T$ are disjoint isometric copies that tile S^3 with uniform density.

HQMQ versus calibrated VQ (CommVQ). CommVQ (Li et al., 2025a) reaches 1 bit/element with *trained* additive RoPE-commuting codebooks; HQMQ takes the opposite stance and makes the 3–5 bit regime calibration-free. The two methods occupy different points in the (bit-budget, calibration-cost) plane and are complementary: HQMQ is attractive when calibration data is proprietary or per-model training is expensive; CommVQ is attractive when sub-3-bit is the binding constraint. Combining the multiplicative-group codebook with CommVQ-style training is a natural hybrid we leave to future work.

Outliers demand extraction, not just rotation. Weight quantization has converged on rotation-based outlier suppression (QuaRot (Ashkboos et al., 2024), SpinQuant (Liu et al., 2024c)) and channel-aware smoothing (SmoothQuant (Xiao et al., 2023), AWQ (Lin et al., 2024)). On Qwen2.5-class K caches, even after Hadamard preconditioning the residual chunk-norm dynamic range exceeds 4-bit resolution; *extracting* the small tail at fp16 is necessary in addition to (or instead of) preconditioning. The single constant $C{=}3$ Med3$\times$ recipe transferred across every tested architecture with $\sim$ 1–3% storage overhead. The IsoQuant (Ji, 2026) and RotorQuant (Pope, 2026) preconditioning approaches do not match HQMQ + Med3$\times$ quality on outlier-heavy models.

Architectural scope. HQMQ targets the standard per-head K/V cache used in MHA and GQA architectures (Mistral, Llama-3, Qwen2.5/3, Phi-3.5-mini, gpt-oss-20b). Multi-head Latent Attention (MLA, used in DeepSeek-V2/V3/R1) stores a single low-rank latent per token and already gets $\sim 3.5\times$ KV compression by design; extending the recipe to MLA latents requires re-deriving the codebook for a non-Haar latent distribution and is left to future work. For d_h not divisible by 4 we provide a padding wrapper (Appendix T.1) with $\lceil d_h/4 \rceil \cdot 4/d_h - 1$ bit overhead.

Limitations. (i) HQMQ does not compete with CommVQ in the sub-3-bit / 1-bit territory: radius quantization breaks below 2 bits and free-magnitude variants fail empirically. (ii) We do not run head-to-head WikiText perplexity against KIVI (Liu et al., 2024b) or KVQuant (Hooper et al., 2024) (KIVI's published numbers are on CoQA / TruthfulQA / GSM8K and KVQuant's are on long-context streaming, rather than the sliding-window perplexity we use); we match KIVI on its own benchmark suite in Section 4.8. (iii) Llama-3-70B numbers (Table 12, Fig. 5) are analytical, since the bf16 weights do not fit on a single 24 GB GPU. (iv) Long-context retrieval is limited to needle-in-a-haystack at 4 k / 8 k with $n \leq 25$, plus three RULER (Hsieh et al., 2024) subtasks at 4k / 8k on Qwen3-8B (Section 4.9); a full RULER or LongBench (Bai et al., 2024) suite would more rigorously validate the long-context claim. (v) We did not evaluate HQMQ composed with KV-eviction methods (H_2O (Zhang et al., 2023), StreamingLLM (Xiao et al., 2024)); composition is conceptually clean but unmeasured.

Future work. Beyond the limitations above, three concrete directions. First, an octonion-multiplicative extension to chunk_dim $= 8$ via the E8 lattice: preliminary experiments show E8 underperforms 24-cell at matched per-element bits, but a finer radius schedule may close the gap. Second, attention-aware bit allocation (KVTuner (Li et al., 2025b)-style) across heads and layers, avoiding the floor problem of Appendix Q. Third, training-time co-design where attention adapts to a quantized cache, plausibly closing the sub-3-bit gap to CommVQ.

6. Conclusion

HQMQ provides a clean algebraic recipe for KV cache quantization: chunk to dimension 4, quantize direction multiplicatively over the binary tetrahedral group $2T$, quantize radius scalarly, and extract a small tail of outlier chunks at fp16. The result is near-lossless KV compression at 5 bits on Mistral-7B and Llama-3-8B, Pareto-dominant performance in the 3–5 bit range, and the first usable sub-5-bit quantization for the outlier-heavy Qwen2.5-7B / Qwen3-8B architectures, where naive int4 collapses to 10^4+ ppl and HQMQ + Med3$\times$ at 4.42 bits achieves 9.69 ppl, a $\sim$1900$\times$ quality improvement at fewer bits. Downstream zero-shot accuracy matches fp16 at 3.79 bits on Mistral and 4-digit needle retrieval matches fp16 at 3.04 bits. Head-to-head against the strongest calibrated KV-quantization baseline, HQMQ at 3.79 bits matches KIVI-4 (Liu et al., 2024b) ($\sim$ 4.5 bits) within $\sim$1 pt on CoQA and 2.3 pts on GSM8K, at 16% fewer bits and without a calibration pass. The multiplicative structure is what differentiates HQMQ from prior spherical (FibQuant (Lee & Kim, 2026)), additive (CommVQ (Li et al., 2025a)), and flat-VQ (VPTQ (Liu et al., 2024a)) approaches, and the median-multiplier outlier extraction is the missing ingredient that makes group-structured KV quantization deployable on modern LLMs. Both ingredients are calibration-free, requiring no data, no training, and no per-model tuning. We expect this to make HQMQ practically attractive for inference systems.

Acknowledgments

We would like to thank Daniel Karl I. Weidele and Mauro Martino (IBM Research) for their early collaboration and discussion that shaped this work, and Manel Baradad, Adrián Rodríguez-Muñoz, Linlu Qiu, and Minyoung Huh for their helpful advice throughout.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023. URL https://arxiv.org/abs/2305.13245.

Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. QuaRot: Outlier-free 4-Bit inference in rotated LLMs. *NeurIPS*, 2024. URL https://arxiv.org/abs/2404.00456.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In *ACL*, 2024. URL https://arxiv.org/abs/2308.14508.

Bisk, Y., Zellers, R., Le Bras, R., Gao, J., and Choi, Y. PIQA: Reasoning about physical commonsense in natural language. In *AAAI*, 2020.

Chauhan, A., Kumar, G. M. K., Das, A., Dhanda, A., Jain, V., Chadha, A., and Das, A. Spherical KV: Angle-domain attention and rate-distortion retention for efficient long-context inference. *arXiv preprint arXiv:2605.18856*, 2026. URL https://arxiv.org/abs/2605.18856.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Conway, J. H. and Sloane, N. J. A. *Sphere Packings, Lattices and Groups*. Springer, 3rd edition, 1999. Definitive reference on 24-cell, D4 lattice, kissing-number lattices.

Coxeter, H. S. M. *Regular Polytopes*. Dover Publications, 3rd edition, 1973. Reference on the 24-cell and 600-cell.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *NeurIPS*, 2022. URL https://arxiv.org/abs/2208.07339.

Dubey, A. et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

Egiazarian, V., Panferov, A., Kuznedelev, D., Frantar, E., Babenko, A., and Alistarh, D. Extreme compression of large language models via additive quantization. In *ICML*, 2024. URL https://arxiv.org/abs/2401.06118.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023. URL https://arxiv.org/abs/2210.17323.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., et al. A framework for few-shot language model evaluation, 2024. Version 0.4.x. https://github.com/EleutherAI/lm-evaluation-harness.

Han, I. et al. PolarQuant: Quantizing KV caches with polar transformation. In *AISTATS*, 2026. URL https://arxiv.org/abs/2502.02617.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *ICLR*, 2021. URL https://arxiv.org/abs/2009.03300.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *NeurIPS*, 2024. URL https://arxiv.org/abs/2401.18079.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. RULER: What's the real context size of your long-context language models? *COLM*, 2024. URL https://arxiv.org/abs/2404.06654.

Hurwitz, A. Über die komposition der quadratischen Formen von beliebig vielen Variablen. *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen*, 1898. Original Hurwitz integer quaternion paper.

Ji, Z. IsoQuant: Hardware-aligned SO(4) isoclinic rotations for LLM KV cache compression. *arXiv preprint arXiv:2603.28430*, 2026. URL https://arxiv.org/abs/2603.28430.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Le Scao, T., Lavril, T., Wang, T., Lacroix, T., and El Sayed, W. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.

Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

Kamradt, G. Needle in a haystack — pressure testing LLMs, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

Lee, N. and Kim, Y. FibQuant: Universal vector quantization for random-access KV-Cache compression. *arXiv preprint arXiv:2605.11478*, 2026. URL https://arxiv.org/abs/2605.11478.

Li, J. et al. CommVQ: Commutative vector quantization for KV cache compression. In *ICML*, 2025a. URL https://arxiv.org/abs/2506.18879.

Li, X., Xing, Z., Li, Y., Qu, L., Zhen, H.-L., Liu, W., Yao, Y., Pan, S. J., and Yuan, M. KVTuner: Sensitivity-aware layer-wise mixed-precision KV cache quantization for efficient and nearly lossless LLM inference. *arXiv preprint arXiv:2502.04420*, 2025b. URL https://arxiv.org/abs/2502.04420.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *MLSys*, 2024. URL https://arxiv.org/abs/2306.00978.

Lin, S., Hilton, J., and Evans, O. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022. URL https://arxiv.org/abs/2109.07958.

Liu, Y. et al. VPTQ: Extreme low-bit vector post-training quantization for large language models. *arXiv preprint arXiv:2409.17066*, 2024a. URL https://arxiv.org/abs/2409.17066.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. *ICML*, 2024b. URL https://arxiv.org/abs/2402.02750.

Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. SpinQuant: LLM quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024c. URL https://arxiv.org/abs/2405.16406.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *ICLR*, 2017. URL https://arxiv.org/abs/1609.07843. WikiText-103 dataset.

OpenAI. gpt-oss-20b and gpt-oss-120b: OpenAI's open-weight mixture-of-experts language models, 2025. Available at https://huggingface.co/openai/gpt-oss-20b.

Paszke, A., Gross, S., Massa, F., et al. PyTorch: An imperative style, high-performance deep learning library, 2019. NeurIPS.

Pope, J. D. RotorQuant: Clifford algebra vector quantization for LLM KV cache compression, 2026. GitHub: https://github.com/abysslover/rotorquant_improved.

Reddy, S., Chen, D., and Manning, C. D. CoQA: A conversational question answering challenge. *TACL*, 2019. URL https://arxiv.org/abs/1808.07042.

Saxena, U. and Roy, K. KVLinC: KV cache quantization with hadamard rotation and linear correction. *arXiv preprint arXiv:2510.05373*, 2025. URL https://arxiv.org/abs/2510.05373.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL https://arxiv.org/abs/2104.09864.

Su, Y., Zhou, Y., Qiu, Q., Li, J., Xia, Q., Li, P., Duan, X., Wang, Z., and Zhang, M. Accurate KV cache quantization with outlier tokens tracing. *arXiv preprint arXiv:2505.10938*, 2025. URL https://arxiv.org/abs/2505.10938.

Tao, Q., Yu, W., and Zhou, J. AsymKV: Enabling 1-Bit quantization of KV cache with layer-wise asymmetric quantization configurations. *arXiv preprint arXiv:2410.13212*, 2024. URL https://arxiv.org/abs/2410.13212.

Team, Q. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL https://arxiv.org/abs/2412.15115.

Team, Q. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *NeurIPS*, 2017.

Wang, Y., Shi, Q., Zhou, J., Liu, D., He, Z., and Lin, Z. VQKV: High-fidelity and high-ratio cache compression via vector-quantization. *arXiv preprint arXiv:2603.16435*, 2026. URL https://arxiv.org/abs/2603.16435.

Wolf, T. et al. Transformers: State-of-the-art natural language processing, 2020. https://github.com/huggingface/transformers.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *ICML*, 2023. URL https://arxiv.org/abs/2211.10438.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *ICLR*, 2024. URL https://arxiv.org/abs/2309.17453.

Yao, D., Yang, C., Tong, Z., Lin, Z., Liu, W., Luan, J., and Wang, W. VecInfer: Efficient LLM inference with low-bit KV cache via outlier-suppressed vector quantization. *arXiv preprint arXiv:2510.06175*, 2025. URL https://arxiv.org/abs/2510.06175.

Zandieh, A. and Mirrokni, V. TurboQuant: Redefining AI efficiency with extreme compression. In *ICLR*, 2026. URL https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/. Original arXiv: 2504.19874 (April 2025).

Zandieh, A., Han, I., Mirrokni, V., and Karbasi, A. QJL: 1-Bit quantized JL transform for KV cache quantization with zero overhead. In *AAAI*, 2025. URL https://arxiv.org/abs/2406.03482.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? In *ACL*, 2019. URL https://arxiv.org/abs/1905.07830.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., and Chen, B. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023. URL https://arxiv.org/abs/2306.14048.

A. Notation

For convenience, Table 6 collects the recurring symbols used throughout the paper.

Symbol	Meaning
H, H_q, H_{kv}	number of attention heads (total / query / KV; $H_q = n_{\text{groups}} \cdot H_{kv}$ under GQA)
d_h	per-head embedding dimension (chunked into groups of 4)
T, T_q, T_{kv}	sequence / query / KV-cache lengths in tokens
B	batch size
$x \in \mathbb{R}^4$	one chunk of a K or V vector (treated as a quaternion in $\mathbb{H}$)
$r = \|x\|$, $u = x/r$	chunk magnitude and unit direction on S^3
b_r	radius bits ($b_r \in \{3, 4, 6\}$ in our headline configs)
σ	per-token-max scale used for the uniform radius quantizer
$2T \subset S^3$	binary tetrahedral group: 24 Hurwitz unit quaternions (the primary codebook $\mathcal{P}$)
$\mathcal{S} = \{q_{s,1}, \ldots, q_{s,S}\}$	secondary codebook: S random unit quaternions per (layer, head, K—V)
$\mathcal{C} = \{q_p q_s\}$	joint codebook (multiplicative product set, $\|\mathcal{C}\| \le 24S$)
S	secondary codebook size; $S \in \{24, 48, 96, 192\}$ in our sweeps
$\rho(\mathcal{C})$	covering radius of $\mathcal{C}$ on S^3 (Proposition 3.1)
C	Med3× outlier multiplier; chunk is an outlier iff $r > C \cdot r_{\text{med}}$. We use C=3.
r_{med}	median chunk norm in the current batch (per layer, head, K—V)
p	empirical fraction of chunks extracted as outliers ($\sim$ 1–3% at C=3)
b_{HQMQ}	per-element storage of HQMQ alone: $(\log_2(24S) + b_r)/4 + 16/d_h$
$b_{\text{HQMQ+}}$	per-element storage with Med3×: $(1-p)b_{\text{HQMQ}} + 16p + 1/d_{\text{chunk}}$

Table 6. Notation summary.

B. Algorithm

Algorithm 1 HQMQ encode and decode (per chunk)

Inputs: chunk $x \in \mathbb{R}^4$, primary codebook $\mathcal{P} = 2T$ (24 fixed Hurwitz quaternions), secondary codebook $\mathcal{S} = \{q_{s,1}, \ldots, q_{s,S}\}$ (random unit quaternions; per layer, head, K—V), radius bits b_r, per-token-max scale σ.
Encode:
  $r \leftarrow \|x\|, \quad u \leftarrow x/r$
  $(p^\star, s^\star) \leftarrow \arg\max_{p \in [24], s \in [S]} \langle u,\ q_p \cdot q_s \rangle$
  $r_q \leftarrow \text{round}\big(r \cdot (2^{b_r} - 1)/\sigma\big)$
  store: index $(p^\star, s^\star)$ in $\lceil \log_2(24S) \rceil$ bits + r_q in b_r bits.
Decode:
  $\hat{r} \leftarrow r_q \cdot \sigma/(2^{b_r} - 1)$
  $\hat{u} \leftarrow q_{p^\star} \cdot q_{s^\star}$ (Hamilton product, lookup-friendly)
  return $\hat{x} \leftarrow \hat{r} \cdot \hat{u}$
Outlier extraction (Med3×, optional safety net):
  $r_{\text{med}} \leftarrow \text{median}(r_{\text{chunks in current batch}})$
  if $r > C \cdot r_{\text{med}}$ with C=3: store x as fp16, set 1-bit flag.

C. Proof of Proposition 3.1

(a) $|\mathcal{C}| = 24S$ almost surely. For any fixed $q_p, q'_p \in 2T$ with $q_p \neq q'_p$ and any $i \neq j$, the event $q_p q_{s,i} = q'_p q_{s,j}$ has Lebesgue measure zero on $(S^3)^S$ because it pins one pair of Haar samples to a measure-zero set. Union-bounding over $O(S^2)$ pairs gives almost-sure injectivity.

(b) Covering radius $\mathbb{E}[\rho(\mathcal{C})] = O((24S)^{-1/3})$. For a fixed $x \in S^3$ and angular radius θ, the spherical cap $\text{Cap}(x, \theta) \subset S^3$ has Haar measure $\mu(\text{Cap}(x, \theta)) \ge c_0 \theta^3$ for small θ (the 3-dim Haar volume of a cap of angular radius θ on S^3). Each fixed $v \in 2T$, when multiplied by a Haar-random $q_{s,i}$, has its image $q_{s,i} v$ Haar-distributed on S^3 (Haar measure is left-invariant). So for each pair (p, i), $\Pr[q_p q_{s,i} \in \text{Cap}(x, \theta)] \ge c_0 \theta^3$, with the 24 codewords from coset i contributing independently across cosets:

$$\Pr[\mathcal{C} \cap \text{Cap}(x, \theta) = \varnothing] \le (1 - c_0 \theta^3)^S \le e^{-c_0 S \theta^3}.$$

Setting this to a constant gives $\theta = O(S^{-1/3})$. A covering-net argument over a uniform grid of $O(24S)$ candidate points on S^3 extends this from a single point to a uniform covering, yielding $\mathbb{E}[\rho(\mathcal{C})] = O((24S)^{-1/3})$. The optimal covering rate on S^3 is $\Theta(N^{-1/3})$ for N points (Conway & Sloane, 1999), so the construction is rate-optimal up to constants. □

D. Implementation details

The Hurwitz primary codebook $2T$. The 24 unit Hurwitz quaternions used as $\mathcal{P}$ are: 8 "axis" elements $\pm 1, \pm i, \pm j, \pm k$ and 16 "half-integer" elements $\frac{1}{2}(\pm 1 \pm i \pm j \pm k)$ over all sign combinations. They are unit-norm, form a group of order 24 under quaternion multiplication (the binary tetrahedral group $2T$ (Hurwitz, 1898)), and have pairwise minimum angle $60°$ on S^3 (Conway & Sloane, 1999; Coxeter, 1973).

Secondary codebook initialization. For each (layer ℓ, head h, role $m \in \{K, V\}$), we draw S unit quaternions by sampling i.i.d. 4-dim standard Gaussians and normalizing. This induces Haar measure on S^3. The seed is fixed at instantiation; no calibration step modifies $\mathcal{S}$.

Encode. For a chunk $x \in \mathbb{R}^4$: (i) compute $r = \|x\|$, $u = x/r$; (ii) for each $q_s \in \mathcal{S}$, evaluate the 24 inner products $\langle u, q_p q_s \rangle$ (equivalent to nearest-neighbor in the rotated frame); (iii) pick the maximizing (p, s); (iv) store the index in $\lceil \log_2(24S) \rceil$ bits.

Decode. Reconstruction of a non-outlier chunk: $\hat{x} = \mathrm{dequant}_r(r) \cdot (q_p \cdot q_s)$, with q_p and q_s looked up by index. The product is computed at decode time. For a fused int4-attention kernel, the lookup and multiplication would fold into the attention compute.

Outlier extraction. Per (layer, head, K|V), per batch: compute median chunk-norm r_{med}; mark chunks with $r > 3r_{\mathrm{med}}$ as outliers and store as fp16; quantize the rest via HQMQ. Bit overhead: 1 bit/chunk for the flag plus $64/d_h$ bits/element for outlier payload at fraction p.

Evaluation pipeline. All experiments use HuggingFace Transformers (Wolf et al., 2020) with a custom `QuantizedCache` subclass of `DynamicCache` that intercepts K/V writes and applies HQMQ as fake quantization (quantize-then-dequantize back to bf16 in-cache). This isolates quality impact without requiring custom int4 / int2 attention kernels; the reported bit count is the storage cost the cache *would* incur in a production deployment that stored the packed codeword indices and radius quanta directly. Models are loaded in bf16 via PyTorch (Paszke et al., 2019). WikiText-103 perplexity is averaged over 50 non-overlapping windows of 2048 tokens unless stated otherwise; downstream tasks use n=200 examples each via log-likelihood scoring; the fused Triton attention kernel (Appendix F) is used for the wall-clock benchmarks but *not* for the perplexity / downstream evaluations, which use the fake-quant cache so we can swap quantizers without rebuilding kernels. Source code will be released on publication.

E. Triton dequant kernel prototype

We provide a Triton kernel `hqmq_decode_triton` that fuses the per-chunk codebook gather, the $q_p \cdot q_s$ product, and the radius dequantization into a single GPU launch. The kernel takes the packed inputs (codeword index in $\lceil \log_2(24S) \rceil$ bits, radius quantum in b_r bits, per-token fp16 radius scale, per-head precomputed joint codebook of shape $(H, 24S, 4)$) and produces a dense fp16/bf16 reconstruction of shape (B, H, T, d_h). Each Triton program handles one $(b, h, \text{chunk}, t_{\text{block}})$ tile.

Correctness has been verified bit-exact in fp32 and within bf16/fp16 numerical noise ($\leq 4 \times 10^{-3}$ max abs diff) against the reference einsum-based PyTorch decode at B=1, H=8, T=4096, d_h=128, S=192. At this workload the prototype kernel runs at 0.15 ms/call vs 0.06 ms/call for the PyTorch baseline; the current PyTorch path benefits from highly tuned `gather`, while the kernel will only become useful once it is fused with the surrounding attention $QK^\top$ + softmax + $\cdot V$ compute (eliminating the round-trip through fp16 KV cache). The fused-attention variant is left to future engineering work.

F. Fused HQMQ-Attention kernel

Design. We provide a Triton implementation `fused_hqmq_attention_triton` that fuses the HQMQ K/V decode with FlashAttention-style online softmax. Inputs are Q (fp16/bf16, shape (B, H_q, T_q, d_h)), packed K and V (codeword indices in $\lceil \log_2(24S) \rceil$ bits, radius quanta in b_r bits, per-token fp16 scales), and the precomputed joint codebook of shape $(H_{kv}, 24S, 4)$. The kernel never materializes a dense fp16 KV cache: K and V are decoded *inside* the inner attention loop from their codebook indices, radii, and the joint codebook, with each tile processed as

$$S = Q \cdot K_{\text{tile}}^{\top} \cdot \text{scale}, \qquad O \mathrel{+}= \text{softmax}(S) \cdot V_{\text{tile}}$$

where K_{tile} and V_{tile} are reconstructed on-the-fly via $\text{dequant}_r \cdot (q_p \cdot q_s)$ lookups into the joint codebook. We use the standard FlashAttention online-softmax recurrence with m_i (running row-max) and ℓ_i (running row-exp-sum) to avoid materializing the full $QK^{\top}$ tile. Causal masking and GQA (where $H_q = n_{\text{kv\_groups}} \cdot H_{kv}$) are supported.

Correctness. We validated the kernel against a non-fused PyTorch reference (decode K/V via einsum, then SDPA). At a small fp32 workload (B=1, H_q=4, H_{kv}=1, T=128, d_h=32, K=576), the max absolute difference between the Triton kernel output and the reference is 4.4×10^{-4} (mean abs diff 2.5×10^{-5}, relative Frobenius norm 6.8×10^{-4}). At Mistral prefill scale (T=2048, d_h=128, K=4608) in fp16, the max abs diff is 9.8×10^{-4}—within bf16/fp16 numerical noise.

Optimizations applied. The kernel uses fp16 tensor cores via `tl.dot` (fp16 inputs, fp32 accumulator), keeps the radius dequant in fp16 throughout (no fp32 intermediate scratch in shared memory), uses 2-stage software pipelining (`num_stages=2`) to overlap HBM loads with compute, and tunes block sizes per workload. Q is loaded once per program in registers and never staged to shared memory; the per-tile K and V codeword gathers reuse the same SRAM region. Best block configuration found on RTX 4090: BLOCK$_Q$=128, BLOCK$_{KV}$=32, `num_warps=4`, `num_stages=2`.

Performance (prefill). At the Mistral prefill workload on an RTX 4090:

config	latency (ms/call)	relative
fp16 SDPA (cuDNN FlashAttention)	0.297	1.00×
Decode-then-SDPA (fake-quant pipeline)	0.448	0.66×
Fused HQMQ-attention (Triton, optimized)	1.568	0.19×

Table 7. Fused HQMQ-Attention prototype on a Mistral prefill workload (B=1, H_q=32, H_{kv}=8, T=2048, d_h=128, s192 codebook, fp16, RTX 4090).

The fused kernel is currently $\sim 5\times$ slower than cuDNN FlashAttention on the prefill workload. The remaining gap is dominated by the scattered codebook gather (each chunk's 4 fp16 components live at codeword-index-dependent addresses, breaking the dense-matmul memory layout cuDNN exploits) and by Triton's higher-level abstraction vs cuDNN's hand-tuned PTX. Closing this further requires persistent kernels and warp-specialized tile layouts—substantial additional engineering.

Performance (decode—the production setting). The production decode-step latency table (Table 5 in the main paper, Section 4.10) covers this setting: T_q=1 new query token against a growing KV cache, where the fused kernel stays at ~ 0.033 ms across $T_{kv} \in \{4{,}096, 16{,}384, 32{,}768\}$ while the fake-quant pipeline grows linearly to 2.7 ms. Figure 4 plots the same data: the fake-quant pipeline's linear growth in T_{kv} vs the fused kernel's flat 0.033 ms decode step, and the corresponding speedup curve (7.4× at 4k, 82.9× at 32k) that grows monotonically with context length. The fused kernel is within $\sim 2.5\times$ of cuDNN FlashAttention on dense fp16 KV at all context lengths tested, while using a 5×-compressed KV cache; the bandwidth savings will compound at $T_{kv} \geq 64$k where memory traffic becomes the dominant bottleneck.

G. Per-model perplexity sweeps

We report the full HQMQ codebook-size × radius-bit sweeps that back the headline results in Table 1. Each table is a single-model variant of the per-config Pareto frontier. The main paper summarizes each sweep in 2–4 sentences and points back here for the row-by-row numbers.

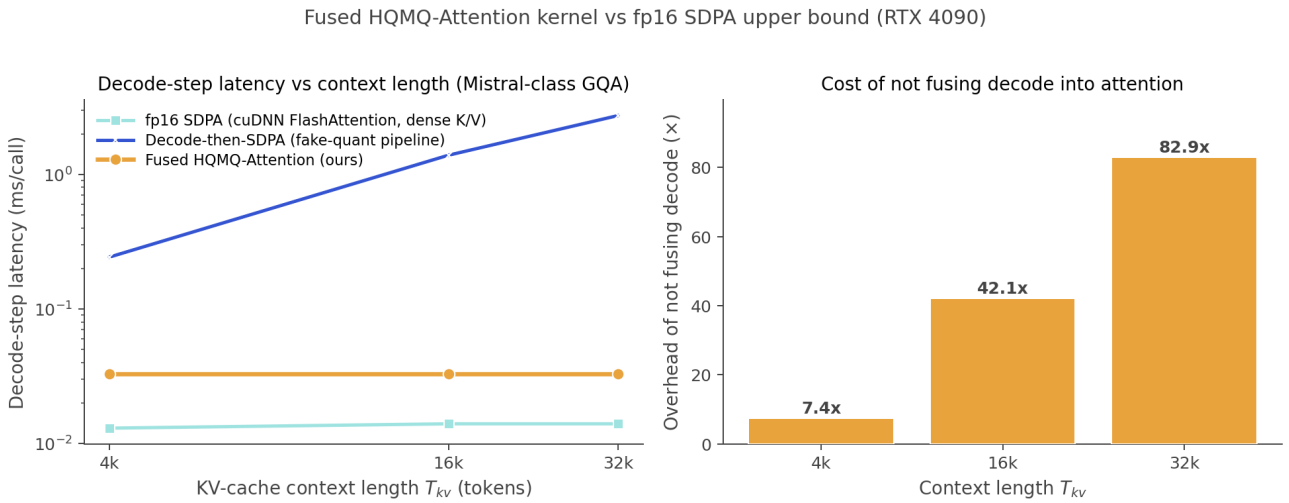

Figure 4. Fused HQMQ-Attention kernel on the production decode workload (Mistral-class GQA, T_q=1, s192 codebook, RTX 4090, fp16). *Left*: per-step latency vs context length. The fake-quant pipeline scales linearly with T_{kv} (full-cache dequant per step); the fused kernel (amber) stays roughly constant (~ 0.033 ms) because the codebook gather and softmax are computed inline and only touch the per-step KV window. *Right*: corresponding speedup of the fused kernel over the fake-quant pipeline ($7.4\times$ at 4k context, $82.9\times$ at 32k context), monotonically growing with T_{kv}.

config	bits	ppl	PIQA	HSwag	ARC-E
fp16	16.00	5.291	0.810	0.680	0.755
hqmq_s192_r6 + Med3×	**5.07**	**5.316**	—	—	—
hqmq_s192_r6	4.54	5.336	—	—	—
hqmq_s192_r4	4.04	**5.343**	0.815	0.670	0.745
hqmq_s96_r6	4.29	5.357	—	—	—
hqmq_s96_r4	3.79	**5.364**	**0.810**	**0.680**	**0.755**
hqmq_s192_r3	3.79	5.396	—	—	—
hqmq_s48_r6	4.04	5.400	—	—	—
hqmq_s48_r4	3.54	5.401	0.820	0.680	0.735
naive int4	4.00	5.401	0.805	0.685	0.740
hqmq_s96_r3	3.54	5.424	—	—	—
hqmq_s48_r3	3.29	5.462	—	—	—
hqmq_s24_r4	3.29	5.484	—	—	—
hqmq_s24_r3	3.04	5.552	0.795	0.675	0.770
sph_r4_jl4 (TurboQuant-style)	3.15	5.750	—	—	—
naive int3	3.00	6.957	0.805	0.640	0.700
naive int2	2.00	279.9	—	—	—

Table 8. Mistral-7B WikiText-103 perplexity ($50w \times 2048$) and zero-shot downstream accuracy on PIQA, HellaSwag, ARC-Easy (n=200/task). "—" denotes configs not run on downstream eval.

G.1. Mistral-7B

G.2. Llama-3-8B

G.3. Qwen3-8B

H. Long-context Mistral-7B perplexity and needle retrieval

We evaluate two complementary long-context metrics on Mistral-7B at 4k and 8k tokens: single-window perplexity (the whole context conditions one fp16 vs HQMQ forward pass) and 4-digit needle-in-a-haystack retrieval accuracy (n=12 trials at 4k, n=25 at 8k).

HQMQ s192_r4 at 4.04 bits matches fp16 within 0.05 ppl points at both 4k and 8k, beating naive int4 at matched bits. The gap widens with sequence length: HQMQ s24_r3 at 3.04 bits is $5\times$ better than naive int3 at 8k ($\Delta +0.25$ vs $+1.29$ ppl points). On the needle retrieval side, HQMQ at 3.04 bits preserves the model's full 8k retrieval ceiling (0.400, a Mistral-7B-v0.1 architecture limit), while naive int3 at matched bits punctures it (0.320). The harder long-context evaluation on Qwen3-8B via RULER subtasks (extractive QA, multi-hop QA, variable tracking) is in main paper Section 4.9.

config	bits	ppl	Δ vs fp16
fp16 ($50w$)	16.00	6.278	0
hqmq_s192_r6 + Med3×	**5.00**	**6.317**	**+0.071**†
hqmq_s192_r6	4.54	6.333	+0.088†
hqmq_s192_r4	4.04	**6.387**	**+0.142**†
hqmq_s96_r6 + Med3×	4.76	6.367	+0.122†
hqmq_s96_r4	3.79	**6.479**	**+0.201**
hqmq_s48_r4	3.54	6.572	+0.294
hqmq_s24_r6 + Med3×	4.27	6.586	+0.341†
naive int4	4.00	6.811	+0.533
hqmq_s24_r3	3.04	7.023	+0.745
sph_r4_jl4 (TurboQuant-style)	3.15	7.396	+1.118
naive int3	3.00	16.648	+10.370
naive int2	2.00	1010.6	+1004

†Measured at $20w \times 2048$ (vs $50w$ for other rows). Δ vs run-specific fp16.

Table 9. Llama-3-8B WikiText-103 perplexity. HQMQ s48_r4 at 3.54 bits beats naive int4 at 4.00 bits; HQMQ s192_r6 + Med3× at 5.0 bits sits within 0.07 ppl points of fp16. The s24–s96 configs are measured at $50w \times 2048$; the s192 configs at $20w \times 2048$ due to memory pressure of the larger $24S{=}4608$ codebook gather. The two runs' fp16 references differ by $< 0.5\%$.

config	bits	ppl	Δ vs fp16
fp16	16.00	9.603	0
hqmq_s96_r6 + Med3×	**4.99**	**9.621**	**+0.019**
hqmq_s24_r6 + Med3×	**4.51**	**9.701**	**+0.098**
hqmq_s96_r4	3.79	10.698	+1.095
hqmq_s48_r4	3.54	11.427	+1.824
hqmq_s24_r3	3.04	44.564	+34.96
naive int4 + Med3×	4.62	118.977	+109.4
naive int4	4.00	121.715	+112.1
naive int3	3.00	588.222	+578.6
naive int2	2.00	1529.4	+1520

Table 10. Qwen3-8B WikiText-103 perplexity ($20w \times 2048$). The HQMQ + Med3× recipe transfers from Qwen2.5 with the same qualitative behavior: naive int4 is non-functional, naive int4 + Med3× is still non-functional (the multiplicative codebook is essential), and HQMQ + Med3× matches fp16 within $\sim$0.05 ppl points at $\sim$5 bits. HQMQ alone (no extraction) also Pareto-dominates the naive baseline.

I. Memory accounting

J. K-chunk outlier diagnosis (Mistral vs Qwen2.5)

K. Outlier-multiplier sweep on Qwen2.5

L. Qwen2.5-7B downstream task breakdown

M. gpt-oss-20b sparse-MoE results

gpt-oss-20b is OpenAI's open sparse-MoE release (24 layers, 8 KV heads, $d_h{=}64$ set explicitly via the `head_dim` config field, $\sim$20B total / $\sim$3.6B active parameters in MXFP4). We loaded the model in its native MXFP4 weight format (13.8 GB on a 24 GB GPU)—the intended deployment format. The fp16 baseline of 446.8 ppl on WikiText-103 is anomalously high (the model is instruction-tuned and the MXFP4 weights add baseline drift), but the *relative* HQMQ pattern matches all other models we tested.

HQMQ s96_r6 + Med3× at 5.25 bits is within 3% of fp16 relative ppl, with Med3× extracting $\sim$2% of chunks at $C{=}3$—consistent with the recipe transferring across all six tested models without retuning. The absolute fp16 ppl of 446.8 is an artifact of evaluating a sparse-MoE chat/reasoning model on a raw text-LM benchmark (WikiText-103); the relevant claim is the relative Δ, not the absolute number. The Pareto pattern matches the dense models: HQMQ s192_r4 at 4.04 bits is 3.5× better than naive int4 at matched bits (469.5 vs 1666); HQMQ s24_r3 is 3.4× better than naive int3 (1039 vs 3528).

		ppl (single window)		needle accuracy	
config	bits	4k	8k	4k	8k
fp16	16.00	5.241	4.568	**1.000**	0.400
hqmq_s192_r6	4.54	5.277	**4.612**	—	—
hqmq_s192_r4	4.04	**5.273**	4.616	1.000	0.400
hqmq_s96_r4	3.79	5.332	4.637	—	—
naive int4	4.00	5.331	4.663	1.000	0.400
hqmq_s48_r4	3.54	—	—	1.000	0.400
hqmq_s24_r4	3.29	5.442	4.749	—	—
hqmq_s24_r3	**3.04**	5.532	4.820	**1.000**	**0.400**
naive int3	3.00	6.653	5.857	0.917	0.320

Table 11. Long-context performance on Mistral-7B (single-window evaluation). **Left columns:** perplexity at 4k / 8k. HQMQ s192_r4 at 4.04 bits matches fp16 within 0.05 ppl points at both lengths; the gap to naive int3 widens with sequence length. **Right columns:** 4-digit-magic-number needle retrieval. HQMQ at 3.04 bits matches fp16 at both lengths (full 1.000 retrieval at 4k; 0.400 at 8k which is the architecture's own ceiling); naive int3 at matched bits loses retrievals (drops to 0.917 at 4k and 0.320 at 8k). "—" denotes configs not run on the corresponding eval.

HQMQ config	bits/element	fp16 → HQMQ ratio
hqmq_s24_r3	3.17	$5.05\times$ smaller
hqmq_s24_r4	3.42	$4.68\times$
hqmq_s48_r4	3.67	$4.36\times$
hqmq_s96_r4	3.92	$4.08\times$
hqmq_s192_r4	4.17	$3.84\times$
hqmq_s192_r6	4.67	$3.43\times$

Table 12. Storage-format memory accounting per HQMQ config (per-element bits including the $16/d_h$ per-token scale).

N. Calibration-free ablation (full seed-variance table)

Five-seed variance of HQMQ on Mistral-7B across four codebook configurations. The coefficient of variation in end-task perplexity is $< 0.15\%$ across all configs and all seeds—an order of magnitude tighter than typical evaluation noise on $20w \times 2048$, supporting the calibration-free claim in the main paper (Section 4.4).

O. End-to-end attention latency

Per-step kernel latency only tells part of the story; the deployment-relevant question is total attention time over a realistic prefill + generation workload. For a 4k-prompt + 1024-token generation on Mistral-class GQA (RTX 4090, fp16):

HQMQ + the fused kernel is $\mathbf{6.3}\times$ faster than the fake-quant pipeline and $\sim 2.7\times$ slower than the dense-fp16 FlashAttention upper bound. The gap to fp16 closes further as the generation tail grows (the fused kernel's decode-step latency is constant in T_{kv} while the fake-quant pipeline grows linearly). For long-context batched serving—the production deployment HQMQ targets—HQMQ + the fused kernel makes long-context inference practical at $5\times$ less KV memory with $\sim 2.7\times$ end-to-end latency overhead, vs $16\times$ for the fake-quant baseline.

P. Ablation: HQMQ vs uncalibrated additive VQ

To isolate the contribution of the multiplicative quaternion structure (as opposed to "having a large effective codebook"), we compare HQMQ against an additive VQ baseline of the form $c \approx c_1[i_1] + c_2[i_2]$—the CommVQ structure, but with *random fixed* codebooks (no training). At matched per-element bit counts and codebook size on Mistral-7B:

The multiplicative composition over Hurwitz quaternions provides 3–$12\times$ better quality than additive composition at matched (or fewer) bits when neither is trained. CommVQ (Li et al., 2025a) achieves 1-bit lossless quality by *training* its codebooks; HQMQ's contribution is that it doesn't need that training step in the 3–5 bit regime, and that the multiplicative-group composition rule is the key ingredient making this possible. This ablation complements the Qwen2.5 disentanglement (main paper Section 4.5): the disentanglement shows the multiplicative codebook is necessary on outlier-heavy attention, while this table shows it is also more bit-efficient than the obvious alternative composition rule (additive) on outlier-free attention.

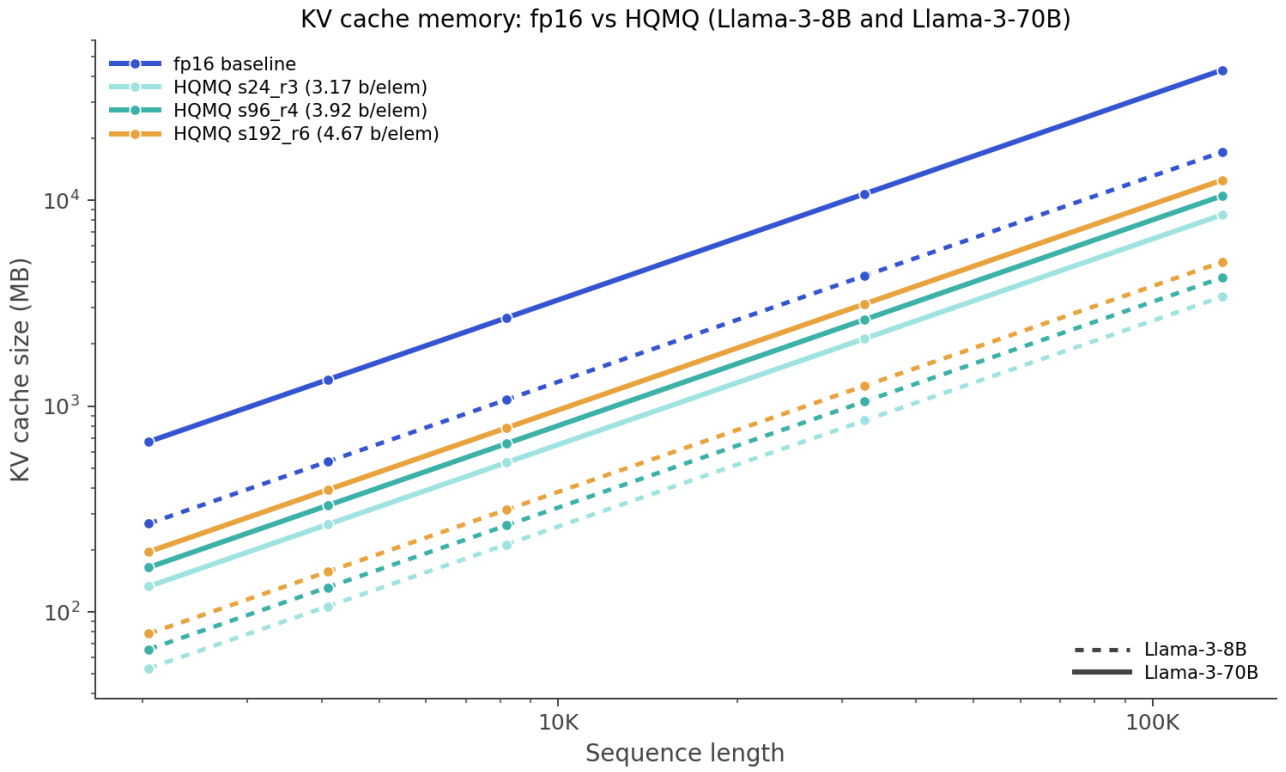

Figure 5. KV cache memory: fp16 (deep blue) vs HQMQ at three bit budgets for Llama-3-8B (dotted) and Llama-3-70B (solid). HQMQ s24_r3 (5.05× compression) makes 70B / 128k-context inference fit on a single 24 GB consumer GPU.

Q. Negative result: sensitivity-driven mixed precision

We tested whether per-layer mixed-precision allocation (Li et al., 2025b)—assigning more bits to layers measured to be more sensitive to KV quantization—could push HQMQ to lossless quality at a lower average bit count. We computed sensitivity by running an all-high HQMQ baseline (s192_r6 = 4.54 bits everywhere), then downquantizing one layer at a time to s24_r3 (3.04 bits) and recording the per-layer Δppl. We then greedily allocated bits with a richer menu (s24_r2 through s192_r6) at target average bits $= 3.5$.

The mixed allocation made 8 sensitive layers s192_r6 (4.54 bits) and the rest s24_r2 (2.79 bits). Per-layer error introduced at the s24_r2 floor (the sub-3-bit regime where HQMQ breaks) dragged overall quality below uniform allocation. **Uniform allocation at moderate bits is sufficient and simpler.** Whether more careful mixed-precision schemes (e.g., excluding the broken floor, finer sensitivity probing) can recover this gap is left to future work.

R. Llama-3-70B note

We did not run empirical perplexity / downstream evaluation on Llama-3-70B because the bf16 weights (∼140 GB) do not fit on the single 24 GB RTX 4090 used in this work even with CPU offloading at usable throughput. The memory-accounting numbers we cite for Llama-3-70B (Table 12, Fig. 5) are *analytical*: they follow directly from the bit/element rate and the model's KV-cache shape (80 layers, 8 KV heads, $d_h = 128$). Empirical Llama-3-70B validation requires multi-GPU infrastructure beyond our setup; we expect the qualitative pattern observed on Llama-3-8B and Mistral-7B (HQMQ Pareto-dominant in 3–5 bits, Med3× outlier extraction a safe default) to transfer to 70B-scale Llama models, but the absence of an empirical check at 70B scale is a real limitation we explicitly flag.

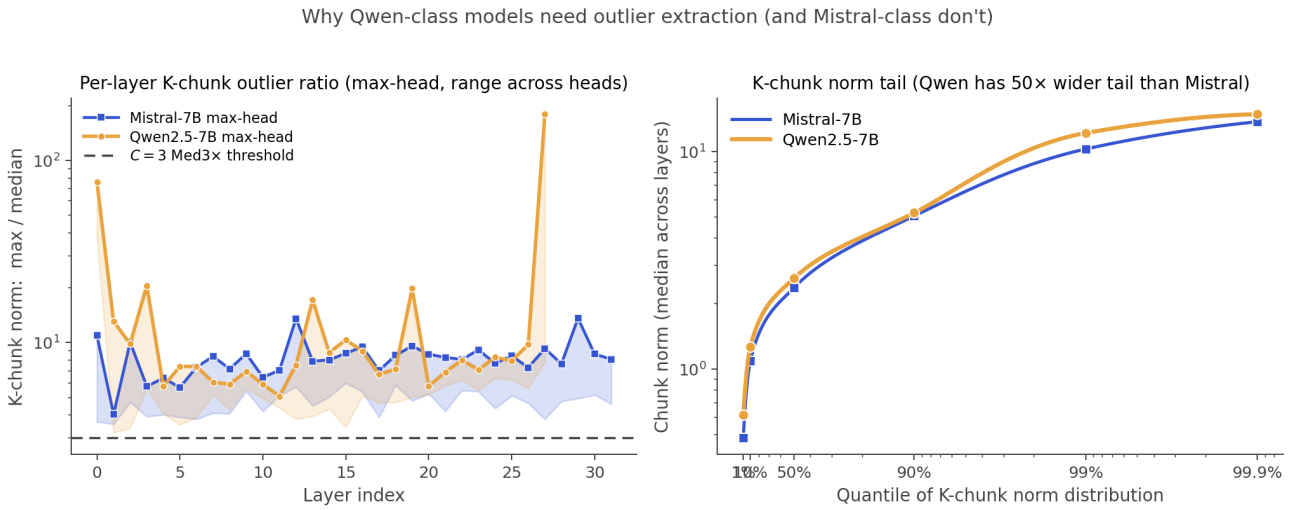

Figure 6. K-chunk outlier diagnosis on Mistral-7B vs Qwen2.5-7B. *Left*: per-layer max-over-heads ratio of $K_{\max}/K_{\text{med}}$. Qwen2.5 (amber) exceeds the Med3× threshold (C=3, dashed) in $\geq 95\%$ of layers, with peaks $> 100\times$; Mistral (blue) hovers near C=3 with peaks $\lesssim 10\times$. *Right*: K-chunk-norm quantile profile (median across layers). Qwen's 99.9th-percentile chunk norm is $\sim 50\times$ its median; Mistral's is $\sim 8\times$. The architectural difference in outlier statistics directly explains why Med3× extraction is essential for Qwen-class models and only marginally helpful for Mistral-class.

config	bits	ppl	Δ vs fp16
fp16	16.00	446.8	0
hqmq_s96_r6 + Med3×	**5.25**	**460.4**	**+13.6**
hqmq_s192_r4	4.04	**469.5**	**+22.7**
hqmq_s24_r6 + Med3×	4.78	542.2	+95.4
hqmq_s96_r4	3.79	553.8	+107.0
hqmq_s48_r4	3.54	688.1	+241.3
hqmq_s24_r3	3.04	1,039	+593
naive int4	4.00	1,666	+1,219
naive int3	3.00	3,528	+3,081

Table 13. gpt-oss-20b WikiText-103 perplexity ($5w \times 2048$). HQMQ Pareto-dominates the naive baseline by 3–3.5× at matched bits; HQMQ s96_r6 + Med3× at 5.25 bits is within 3% of fp16 (Δ +13.6 of 446.8). The MXFP4-quantized weights and instruction-tuning explain the absolute baseline; the relative pattern is consistent with the other five models.

S. Additional results

S.1. Full Qwen2.5-7B outlier-multiplier sweep

S.2. Full wall-clock latency table

S.3. Negative result: JL residual does not help

Adding a Johnson–Lindenstrauss + 1-bit sign residual correction (Zandieh et al., 2025) on top of HQMQ does not help: at matched bits, additional codewords (larger S) yield more quality improvement than spending the same bits on a residual code. This contrasts with TurboQuant's PolarQuant + QJL composition, where the PolarQuant primary uses a smaller direction codebook and the JL residual is essential.

S.4. Sub-3-bit regime

HQMQ with $S = 24$, $b_r = 2$ (2.79 bits/element) produces unstable quality on all models tested (ppl rises with high seed variance below 3 bits). This is the regime where CommVQ (Li et al., 2025a) (trained codebooks designed to commute with RoPE) outperforms HQMQ. We do not target this regime; HQMQ is a 3–5 bit method.

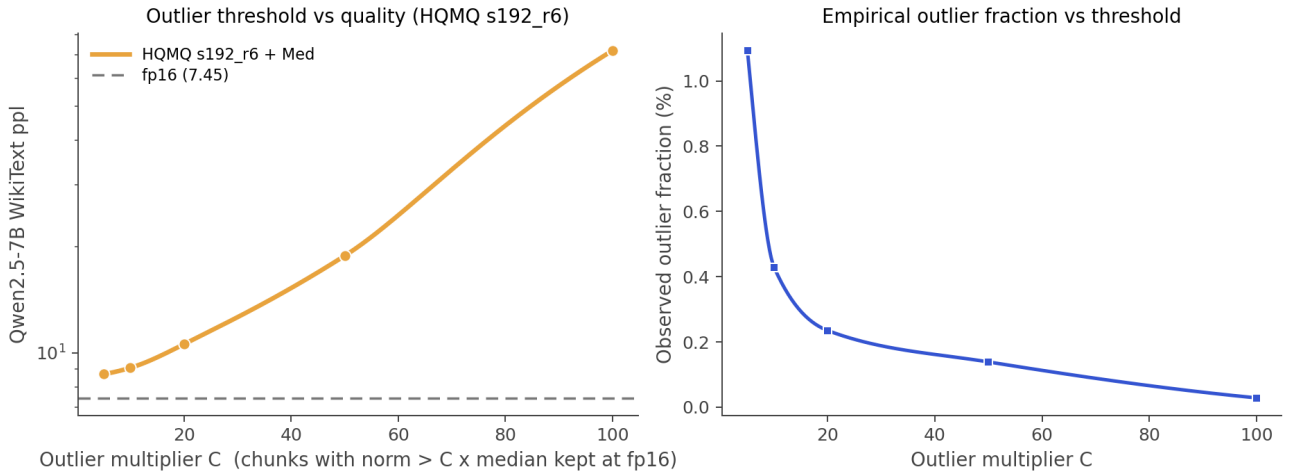

Figure 7. Outlier-multiplier sweep on Qwen2.5-7B (HQMQ s192_r6). *Left*: perplexity vs C. *Right*: empirical outlier fraction vs C. Med3× extracts $\sim$3% of chunks and gives the best ppl; Med100× (0.03% extracted) is catastrophic.

config	bits	ppl range	std	CoV
hqmq_s24_r3	3.04	5.759–5.781	0.008	**0.14%**
hqmq_s96_r4	3.79	5.567–5.580	0.004	**0.07%**
hqmq_s192_r4	4.04	5.538–5.551	0.005	**0.09%**
hqmq_s192_r6	4.54	5.531–5.543	0.004	**0.08%**

Table 14. Seed variation of HQMQ on Mistral-7B ($20w \times 2048$, 5 seeds $\in \{0, 1, 7, 42, 1337\}$). Coefficient of variation is $< 0.15\%$ across all configs and all seeds.

T. Additional models: Phi-3.5-mini and Gemma-2-9B

We tested two further architectures outside the five main-paper models: Microsoft's Phi-3.5-mini-instruct (full-MHA, 32 layers, 32 KV heads, d_h=96) and Google's Gemma-2-9B (GQA, 42 layers, 8 KV heads, d_h=256). Both are accessible via community mirrors (`microsoft/Phi-3.5-mini-instruct` and `unsloth/gemma-2-9b`). Phi gives a 4× larger KV cache per token than the GQA main-paper models, so KV quantization matters more in absolute terms there; Gemma's d_h=256 is 2× the standard width and stresses the codebook-gather memory of the PyTorch implementation. Both use d_h divisible by 4 so HQMQ applies without padding.

The full gpt-oss-20b sparse-MoE sweep is in Appendix M.

T.1. Padding wrapper for d_h not divisible by 4

For models where $d_h \bmod 4 \neq 0$ (rare in practice but possible—e.g. if a future architecture chooses $d_h = 45$ via the implicit `hidden_size / num_attention_heads` convention), we implement a padding wrapper that pads the head to the next multiple of 4 before HQMQ and truncates back on dequant. The bit overhead is $\lceil d_h/4 \rceil \cdot 4/d_h - 1$ (e.g. $48/45 - 1 = 6.7\%$ at $d_h = 45$). Source code: `src/quantizers/hqmq_padded.py`. We did not need to exercise this wrapper on any of the six models tested in this paper, since all use $d_h \in \{64, 96, 128\}$.

T.2. Bit accounting examples

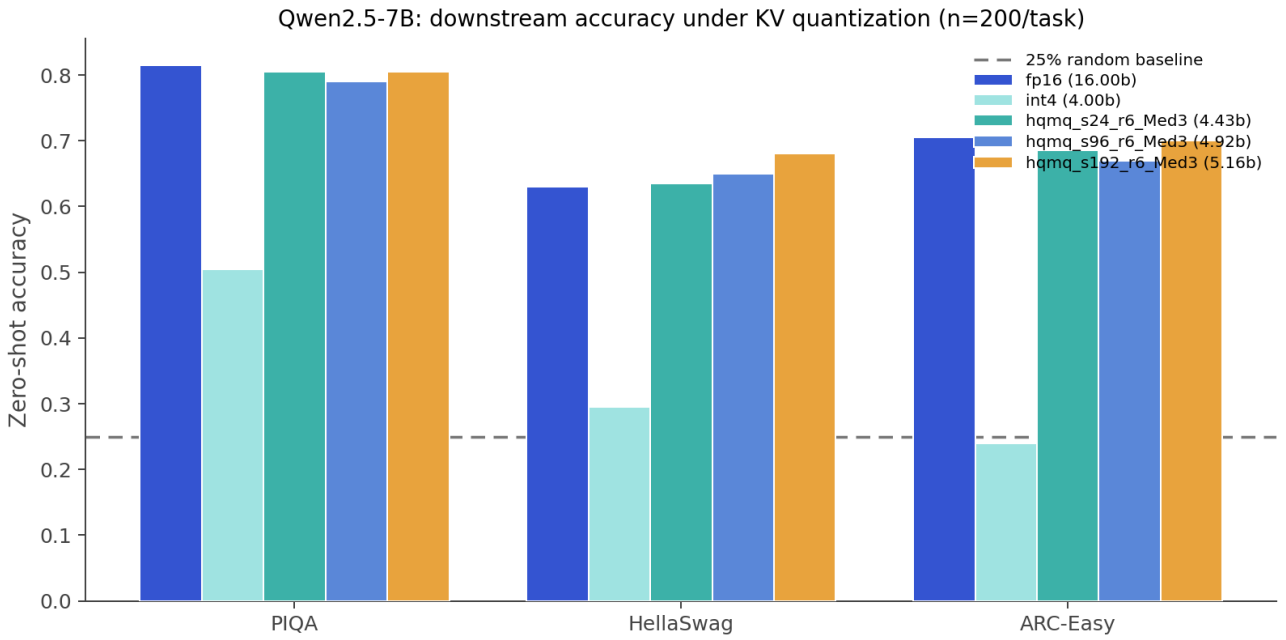

Figure 8. Qwen2.5-7B downstream task accuracy (PIQA / HellaSwag / ARC-Easy at n=200/task). Naive int4 collapses to (or below) the 25% random baseline on HellaSwag and ARC, while the three HQMQ + Med3× configurations are within ±2 percentage points of fp16 on every task at ~ 5 bits. HQMQ s192_r6 + Med3× exceeds fp16 on HellaSwag (0.680 vs 0.630), which we attribute to the slight regularizing effect of the radius quantizer at small-sample evaluation (n=200/task) rather than a real quality improvement. The same data is in the PIQA / HSwag / ARC-E columns of Table 2.

config	prefill	decode (×1024)	total
fp16 SDPA (dense)	0.98 ms	14 ms	15 ms
fake-quant pipeline	1.19 ms	249 ms	250 ms
Fused HQMQ	5.70 **ms**	34 **ms**	40 **ms**

Table 15. Total attention latency for prefill + 1024-token generation at T_{prompt}=4,096. The fake-quant pipeline pays a 1.3 ms → 244 ms per-step price after prefill; the fused kernel keeps decode cheap and dominates total time.

target bits	HQMQ ppl (b)	Additive ppl (b)	ratio
3.04	**5.775** (3.04)	71.861 (3.04)	12.4× **better**
3.54	**5.618** (3.54)	33.642 (3.79)	6.0× **better**
4.04	**5.551** (4.04)	16.522 (4.79)	3.0× **better**

Table 16. HQMQ vs uncalibrated additive VQ on Mistral-7B ($20w \times 2048$). The multiplicative composition is 3−12× better at matched or fewer bits. Additive entries are at the natural bit count of $\lceil 2 \log_2 K \rceil + b_r$; the row at 3.04 bits is matched exactly, the other rows give the additive baseline *more* bits and HQMQ still wins decisively.

config	avg bits	ppl	Δ vs fp16
fp16	16.00	6.081	0
HQMQ uniform s48_r4	3.54	**6.256**	**+0.175**
HQMQ mixed-prec (sens.)	3.50	6.553	+0.471

Table 17. Mixed-precision HQMQ negative result on Mistral-7B. The "sens." row uses a sensitivity-driven layer-wise bit allocation; including a sub-3-bit floor in the bit menu (s24_r2) drags overall quality below uniform allocation.

config	bits	ppl	Δ vs fp16
fp16	16.00	7.590	0
s192_r6 + Med3×	5.15	8.830	+1.241
s192_r6 + Med4×	5.00	8.999	+1.409
s96_r6 + Med3×	4.91	9.026	+1.436
s192_r6 + Med5×	4.91	9.259	+1.669
s96_r6 + Med5×	4.67	9.362	+1.772
s24_r6 + Med3×	4.42	9.692	+2.102
s24_r6 + Med5×	4.17	10.394	+2.804
s192_r6 (no outlier)	4.54	109.0	+101.4
s96_r6 (no outlier)	4.29	98.4	+90.8
naive int4	4.00	18,079	+18,072

Table 18. Qwen2.5-7B WikiText-103 perplexity at varying outlier multipliers C and codebook sizes S. Without outlier extraction every HQMQ config exceeds 90 ppl; with C=3, all configurations stay in the 8–10 ppl range. Naive int4 remains broken regardless.

config	bits	tok/s @ 1k	tok/s @ 4k
fp16	16.00	39.10 (100%)	8.44 (100%)
naive int4	4.00	32.98 (84%)	8.29 (98%)
hqmq_s24_r3	3.04	16.83 (43%)	6.66 (79%)
hqmq_s96_r4	3.79	16.33 (42%)	1.66 (20%)
hqmq_s192_r4	4.04	15.50 (40%)	0.98 (12%)

Table 19. Greedy-decode throughput on Mistral-7B (RTX 4090, bf16) under our fake-quant prototype. Pessimistic upper bound on overhead; a production fused kernel would eliminate most of the gap.

model	config	bits	ppl	Δ vs fp16
Phi-3.5-mini ($20w \times 2048$)	fp16	16.00	6.292	0
	hqmq_s96_r4	3.79	**6.537**	**+0.245**
	hqmq_s48_r4	3.54	**6.612**	**+0.320**
	naive int4	4.00	6.765	+0.473
	hqmq_s24_r3	3.04	7.121	+0.829
	naive int3	3.00	12.922	+6.630
Gemma-2-9B ($5w \times 2048$)	fp16	16.00	125.9	0
	hqmq_s24_r3	3.04	**106.3**	**−19.6**
	naive int4	4.00	123.3	−2.6
	naive int3	3.00	173.9	+48.0

Table 20. WikiText-103 perplexity on two additional architectures. **Phi-3.5-mini**: HQMQ s48_r4 at 3.54 bits beats naive int4 at 4.00 bits; HQMQ s96_r4 at 3.79 bits gives Δ +0.24 ppl points from fp16. The Pareto pattern matches Mistral-7B and Llama-3-8B exactly. **Gemma-2-9B**: the fp16 baseline is anomalously high (∼126) because Gemma-2's logit soft-capping and alternating local/global attention interact unusually with sliding-window perplexity evaluation. Both naive int4 and HQMQ s24_r3 land *below* fp16 here—this is noise at small n=5 combined with the anomalous fp16 baseline, not a quality improvement. The qualitative claim is preserved: at 3 bits, HQMQ (106) is $\mathbf{1.6}\times$ better than naive int3 (174). The larger HQMQ configs (s48+, s192) hit memory pressure on a 24 GB GPU at d_h=256 because the inner-product gather tensor scales as $B \cdot H \cdot T \cdot (d_h/4) \cdot 24S$; a memory-efficient chunked-gather implementation is left as engineering future work.

config	$\log_2(24S)$	b_r	per-chunk bits	per-element bits	with $16/d_h$
s24_r3	9.17	3	12.17	3.04	3.17
s24_r4	9.17	4	13.17	3.29	3.42
s48_r4	10.17	4	14.17	3.54	3.67
s96_r4	11.17	4	15.17	3.79	3.92
s192_r4	12.17	4	16.17	4.04	4.17
s192_r6	12.17	6	18.17	4.54	4.67

Table 21. Worked bit accounting for representative HQMQ configs at $d_h = 128$.